

Surface energy balance models for Evapotranspiration retrieval - TSEB & 3SEB models



Contents

1. Surface energy balance models for Evapotranspiration retrieval - TSEB & 3SEB models	1
1.1 Summary	1
1.2 Local Installation	1
1.3 Run the interactive book	1
1.4 License	2
2. Radiation transfer in canopies	4
2.1 Summary	4
2.2 Instructions	4
2.3 The energy balance	4
2.4 Beer-Lambert law	5
2.5 The albedo	8
2.6 Net shortwave radiation	10
2.7 Shortwave radiation in clumped canopies	11
2.8 Net longwave radiation	14
2.9 Conclusions	15
3. Turbulent exchange of heat and water vapour	17
3.1 Summary	17
3.2 Instructions	17
3.3 Basic equations	17
3.4 The wind profile	18
3.5 The evapotranspiration and the sensible heat flux	22
3.6 About the surface temperature	26
3.7 Conclusions	28
4. How TSEB and 3SEB works?	30
4.1 Summary	30
4.2 Instructions	30
4.3 The TSEB model	30
4.4 Satellite-based implementation of TSEB	41
4.5 Three-Source Energy Balance (3SEB) model adaptation	47
4.6 Conclusions	57
5. Calculation of Shortwave and Longwave Net Radiation	59
5.1 Summary	59
5.2 Instructions	59
5.3 Net shortwave radiation	59
5.4 Net longwave radiation	67
5.5 Net shortwave radiation in 3SEB	69
5.6 Conclusions	73
References	74

1. Surface energy balance models for Evapotranspiration retrieval - TSEB & 3SEB models

1.1 Summary

This repository contains the online material needed to run the digital notebooks (**Jupyter Notebook**) developed for the course.

1.2 Local Installation

In case you want to install and run the notebook locally in your computer:

Prerequisites - Install Python and QGIS

You should have these programs installed:

- Python and/or [Anaconda](#).
- [Git](#)

We need to install all the library requisites for this tutorial.

First navigate in a terminal to the curso-aet folder:

Windows

1. Open the Anaconda Prompt Terminal
2. in the terminal navigate where to the curso-aet folder:

```
cd C:\Users\<user>\curso-aet
```

Linux

1. Open a Terminal (e.g. by presing CTRL+ALT+T)
2. in the terminal navigate where to the curso-aet folder:

```
cd /home/<user>/curso-aet
```

Then install the requirements either with pip or with conda/mamba (recommended):

Pip

```
pip install requirements.txt
```

Conda/Mamba

```
mamba env create -f environment  
conda activate curso-aet
```

1.3 Run the interactive book

Windows

1. Open the Anaconda Prompt Terminal
2. in the terminal navigate where to the curso-aet folder:

```
cd C:\Users\<user>\curso-aet
```

3. Run this command

```
run_workshop.bat
```

Linux

1. Open a Terminal (e.g. by pressing CTRL+ALT+T)
2. in the terminal navigate where to the curso-aet folder:

```
cd /home/<user>/curso-aet
```

3. Run this command

```
bash run_workshop.sh
```

Open your web browser to <http://localhost:3000> .

1.4 License

Creative Commons Attribution-ShareAlike 4.0 International.

This work is licensed under Attribution-ShareAlike 4.0 International. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/>

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, even for commercial purposes. If others remix, adapt, or build upon the material, they must license the modified material under identical terms.

- BY: Credit must be given to you, the creator.
- SA: Adaptations must be shared under the same terms.

2. Radiation transfer in canopies

2.1 Summary

This interactive Jupyter Notebook has the objective of evaluating the different relationships and basic processes related to the interception and absorption of irradiance.

We will make use of simulation models, particularly radiative transfer models at both leaf and canopy levels.

2.2 Instructions

Read carefully all the text and follow the instructions.

Once each section is read, run the jupyter code cell underneath (marked as In []) by clicking the icon Run, or pressing the keys SHIFT+ENTER of your keyboard. A graphical interface will then display, which allows you to interact with and perform the assigned tasks.

To start, please run the following cell to import all the packages required for this notebook. Once you run the cell below, an acknowledgement message, stating all libraries were correctly imported, should be printed on screen.

```
%matplotlib inline
from pathlib import Path
from ipywidgets import interact, interactive, fixed
from IPython.display import display
from functions import radiation_and_available_energy as fn
import numpy as np
```

scikit-learn-intelx not available, skipping the use of Intel acceleration.
Try to consider installing Intel acceleration via `pip install scikit-learn-intelx`

Thanks! libraries imported
You can now continue with the following tasks

2.3 The energy balance

Evapotranspiration is basically an exchange of heat and water between the land surface and the atmosphere. Therefore, the estimation of the energy available at the land surface is one of the key factors affecting its retrieval.

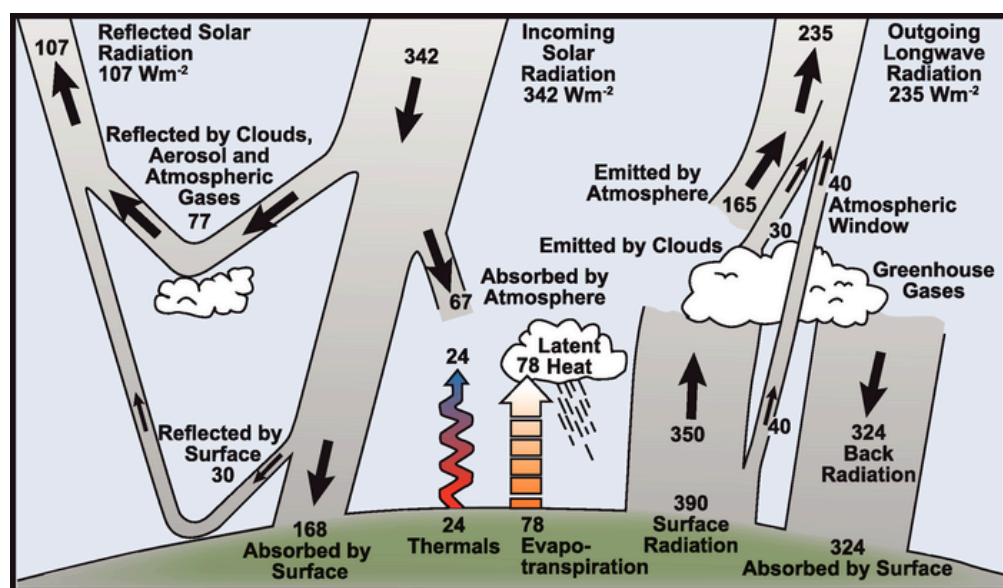


Figure 2.1: The Global energy balance model

Depending on the surface water status, this available energy is utilized for evaporating water (ET or latent heat flux - λE in terms of energy fluxes) and/or for increasing the surface temperature (sensible heat flux - H).

$$\lambda E + H = R_n - G \quad (2.1)$$

where G is the heat transmitted and stored into the ground, and R_n is the net radiation.

The net radiation (R_n) can be computed as:

$$R_n = S^{down}(1 - \alpha) + \varepsilon(L^{down} - \sigma LST^4) \quad (2.2)$$

where S^{down} and L^{down} are the shortwave and longwave incoming irradiances, respectively, which can be obtained from meteorological stations, numerical weather models or meteorological satellites. α and ε are surface albedo and emissivity, respectively, LST is the Land Surface Temperature, and $\sigma \approx 5.67E - 8$ (W/m^2K^4) is the Stefan-Boltzmann constant. α , ε and LST can be estimated, more or less accurately, from Earth Observation (EO) data.

Considering the larger magnitude of shortwave radiance (S) compared to the longwave radiance (L), we will place more emphasis on the former in this tutorial/notebook. In particular, we will concentrate on the canopy and leaf properties that influence albedo and radiation partitioning between soil and canopy.

On the other hand, G is usually of much lower magnitude, usually considered negligible at daily or even longer scales. When working at shorter timescale, e.g. when estimating λE at the satellite overpass time, G is usually estimated as a fraction of surface net radiation (e.g. $G \approx 0.1R_n$ for dense canopies), or as a fraction of net radiation of the soil below the canopy ($G \approx 0.3R_{ns}$).

2.4 Beer-Lambert law

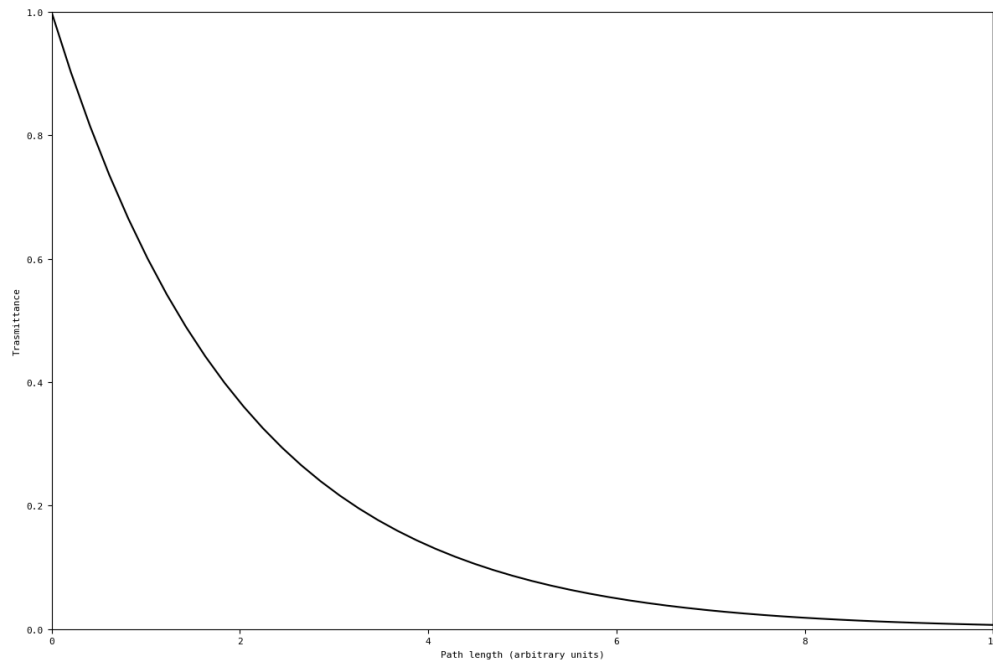
The Beer-Lambert law is a physical law that allows to describe and interpret most of the radiative processes of our interest. When an electromagnetic beam passes through a medium containing an absorber of radiation, this light is attenuated proportionally to the concentration of such absorber (κ) and to the optical path length of the beam (ℓ):

$$\tau = e^{-\kappa\ell} \quad (2.3)$$

In the following interactive graph, you can see the effect of this law. Try modifying the values of the extinction coefficient, which increases with the concentration of the absorbing material in the medium.

The plot will display the fraction of light that is transmitted through the medium as the beam path becomes longer:

```
w_lambert = interactive(fn.beer_lambert_law, kappa=fn.w_kappa,
length=fixed(np.linspace(0, 10, 50)))
display(w_lambert)
```



interactive(children=(FloatSlider(value=0.5, continuous_update=False, description='\$\\kappa\$ (--)', max=1.0, s...

This law permits, for instance, to explain the attenuation of solar radiation from the stratosphere to the Earth's surface. The air is composed of gases and particles that absorb radiation (most of them at certain wavelengths such as the ozone or the oxygen). The higher the sun is in the sky, the shorter the path length across the atmosphere, and, thus, a greater amount of solar irradiance reaches the land surface.

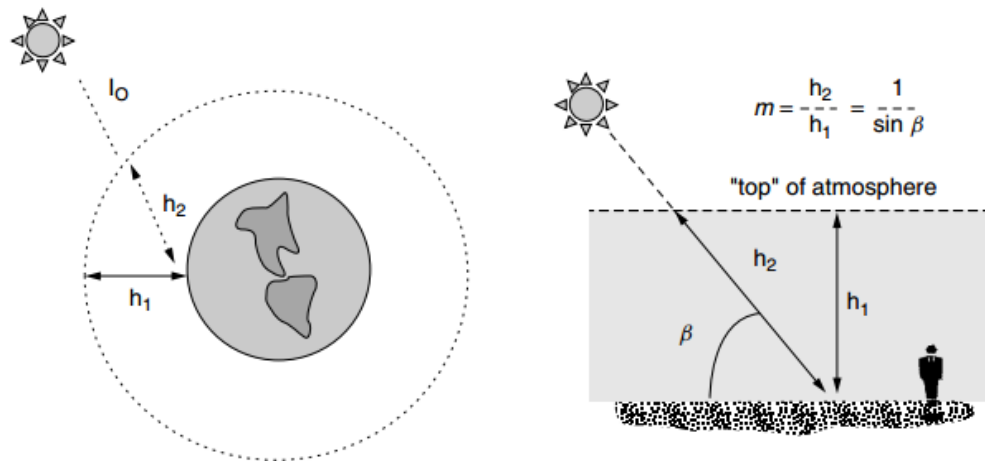


Figure 2.2: The solar optical path length in a sphere (left) and its planar simplification (right)

2.4.1 Application to the fraction of intercepted radiation

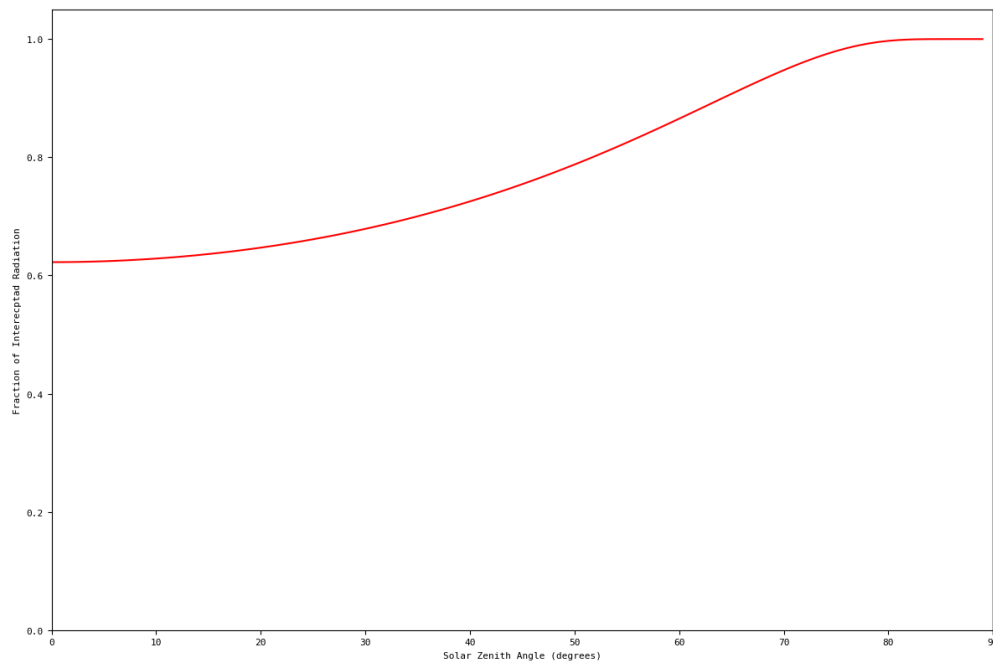
Beer-Lambert law also allows the analysis or estimation of the radiation that is intercepted by the foliage in a canopy, as well as the radiation transmitted towards the ground.

Assuming that a canopy is primarily composed of leaves, and that these leaves intercept and absorb radiation (mostly photosynthetically active radiation, PAR), we can evaluate how much radiation is intercepted by the canopy according to the foliage density. The latter is usually defined as the Leaf Area Index (LAI): half of the total leaf surface per unit ground. The larger the LAI, the more likely that the sun beams will be intercepted by one or more leaves, and, thus less likely for solar beams to reach the ground.

The solar elevation angle (β in the previous graph), or its complementary the solar zenith angle (SZA), also plays a relevant role. The lower the sun is above the horizon, the longer the path of the sun beams will be when passing through the canopy, and thus the larger the light attenuation.

Since leaves are solid elements with a finite size (as opposed to the gases in the atmosphere), their orientation with respect to the nadir also plays a relevant role. Therefore, the Beer-Lambert law must be adapted to account for this effect. In the next graph, you will be able to experiment with this phenomenon. Try changing both the amount of leaves in terms of LAI and the dominant zenith orientation of leaves, i.e. from canopies with predominantly vertical (LIDF $\approx 90^\circ$) leaves to predominantly horizontal (LIDF $\approx 0^\circ$) leaves.

```
w_fipar = interactive(fn.plot_fipar, lai=fn.w_lai, leaf_angle=fn.w_leaf_angle)
display(w_fipar)
```



```
interactive(children=(FloatSlider(value=2.0, continuous_update=False,
description='LAI (m  /m  )', max=4.0, tool...
```

Watch how the largest fraction of intercepted radiation (or the largest light attenuation) always occurs at higher solar zenith angles (when the path length of the solar beams is larger), but that the attenuation rate depends mostly on the amount of leaves (higher LAI) and their dominant orientation.

LAI depends mainly on the plant development stage and plant growth, while the leaf orientation is more specific for the different species or plant functional types.

As a rule of thumb, we could expect that plants with predominantly vertical (i.e. erectophylle) leaves are usually adapted to climates with high irradiance levels, with the aim of avoiding large exposure and interception around noon, when irradiance is maximum. On the other hand, plants with predominantly horizontal leaves tend to thrive in more shaded conditions or with low irradiance levels, in order to maximize the interception of solar radiation along the daytime. Between these two extremes, most of the plants show an angular distribution more or less random. For those cases, the calculation of intercepted radiation can be simplified as:

$$fIPAR = 1 - \exp\left(\frac{-0.5LAI}{\cos \theta_s}\right) \quad (2.4)$$

where θ_s is the sun zenith angle.

See Also

If you want to know more details on these calculations you can check the [pyTSEB GitHub source code](#).

2.5 The albedo

As we have previously mentioned, the albedo (α) is the key variable that determines the amount of intercepted light that is absorbed by the surface. α is defined as the proportion of incident shortwave radiation that is reflected by the surface. The shortwave net radiation (S_n) is therefore the balance between the incident shortwave irradiance (S^{down}) and the reflected shortwave radiance ($S^u_p = \alpha S^{down}$)

The spectral properties of the surface are key in determining the albedo. For instance, fresh snow shows large values ($\alpha \approx 0.9$) and, thus, reflects most of the solar irradiance, while oceans have much lower values ($\alpha \approx 0.05$), absorbing a large amount of solar irradiance.

Leaves, due to their photosynthetic activity, absorb a large proportion of light due to the presence of chlorophylls, as well as other leaf pigments. For that reason, leaves yield relative low albedo values (≈ 0.15). On the other hand, soils can have a large range of albedo values, depending on their mineral composition, texture and topsoil moisture. Therefore, the albedo of a vegetated surface will depend not only on leaf chlorophyll concentration but also on canopy density and, in a lesser degree, on the soil albedo in situations of sparse vegetation or initial growth stages.

2.5.1 Vegetation anisotropy

Most of the Earth's surface show certain anisotropic behaviour when reflecting radiation i.e. it scatters different amounts of radiation depending on the scattering direction. The most extreme cases would be specular surfaces (such as mirrors, still water bodies or ice), which scatter most the irradiance in the opposite direction of the incidence angle. The opposite case would be lambertian surfaces, where apparent brightness/scattering is the same regardless the direction.

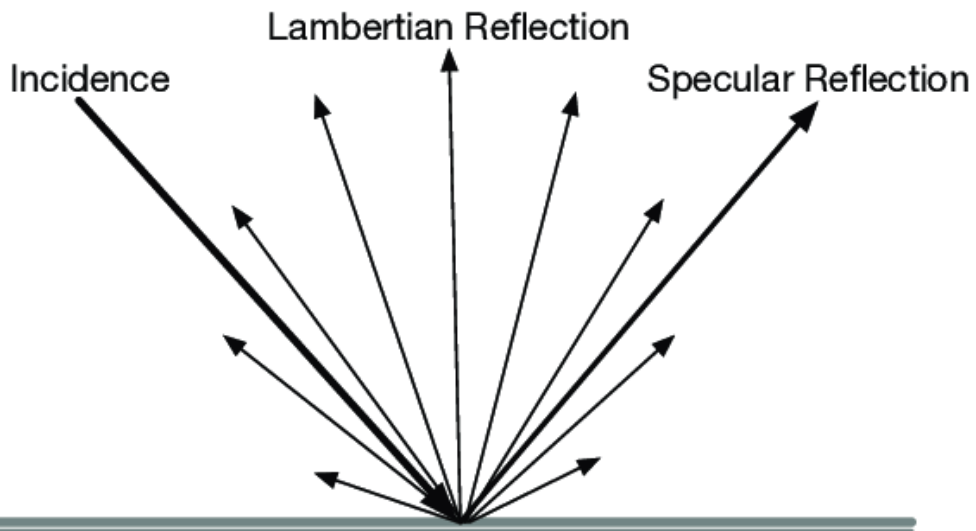


Figure 2.3: The Lambertian (thin arrow lines) and specular (bold arrow line) scattering

Note

Unpolished wood would be closer to a lambertian surface, while polished and varnished wood would behave more as a specular/non-lambertian surface.

Vegetation, as it is mainly composed by an array of leaves, is also affected by this anisotropic behaviour. Therefore, plants will reflect radiation differently depending on the

illumination geometry (i.e. the solar position) and the scattering direction (i.e. the sensor position), possibly changing their albedo.

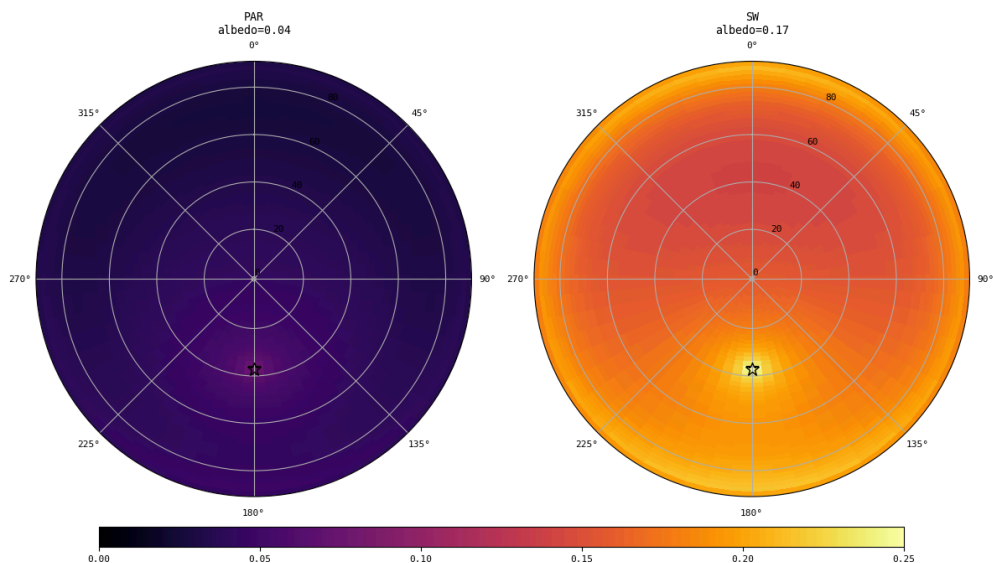
You can visualize this effect in the following plot. It depicts two polar graphs, one for the photosynthetically active radiation (PAR, 400-700 μm) and a second one for the shortwave radiation (SW, 400-2500 μm). These plots represent the reflectance factors simulated for a homogeneous canopy, using the radiative transfer models Féret et al. (2017) and Verhoef et al. (2007).

The concentric rings represent the range of zenith angles (zenith=0° at the plot origin, and 90° at the outermost ring). The radii of the plots show the azimuth angles (0° bearing north, 90° bearing east, 180° bearing south, and 270° bearing west). The solar position is depicted in the graph with a star. By default the sun is placed at solar noon in Northern latitudes (180° azimuth), at a solar zenith angle of 37°. The plots also print the integrated value of the reflectance factors, which is actually the albedo.

See Also

If you want to know more details on these calculations you can check [pypro4sail](#) [GitHub source code](#)

```
w_bidirectional = interactive(fn.bidirectional_reflectance,
                              cab=fn.w_cab, cw=fn.w_cw, lai=fn.w_lai,
                              leaf_angle=fn.w_leaf_angle,
                              sza=fn.w_sza, saa=fn.w_saa, skyl=fn.w_skyl,
                              soil_type=fn.w_soil)
display(w_bidirectional)
```



```
interactive(children=(FloatSlider(value=41.07, continuous_update=False,
description='Cab ( $\mu\text{g}/\text{cm}^2$ )', max=1...
```

- Modify the chlorophyll content - Cab. How do reflectance factors and albedo change in the PAR region (400-700 μm) and in the solar spectrum (400-2500 μm)?
- Likewise, modify the leaf water content- Cw. How do reflectance factors and albedo change in the PAR region (400-700 μm) and in the solar spectrum (400-2500 μm)?
- Modify the solar position - SZA and SAA. On one side, you will see a peak of reflectance around the solar position (the star in the plot). This is the so-called hotspot, or the area most illuminated in the canopy. By contrast, the lowest values are on the opposite side, since leaves in this area are more occluded by the rest of the canopy. The albedo values also vary with the solar position, i.e. albedo changes along the day.
- This variation of albedo with solar position is stronger with lower values of diffuse radiation. Increase the ratio of diffuse radiation skyl, which would simulate cloudy

days. On totally overcast days ($sky_l=1$), you will notice that the solar position is no longer relevant for the albedo or reflectance factors.

2.6 Net shortwave radiation

Previously, we have seen that LAI, leaf angle distribution and chlorophyll concentrations are the probably the most relevant variables that determine the absorption of shortwave irradiance within a crop/vegetation layer.

In the next plot, we see the daily trend of net shortwave radiation for a horizontally homogeneous herbaceous crop (such a wheat field). These simulations assume that the cloudiness remains constant along the day and, thus, shortwave irradiance displays a sinusoidal curve: should cloudiness change along the day, the shortwave irradiance would display a series of valleys and peaks, according to whether the sun is occluded by clouds or not.

Important

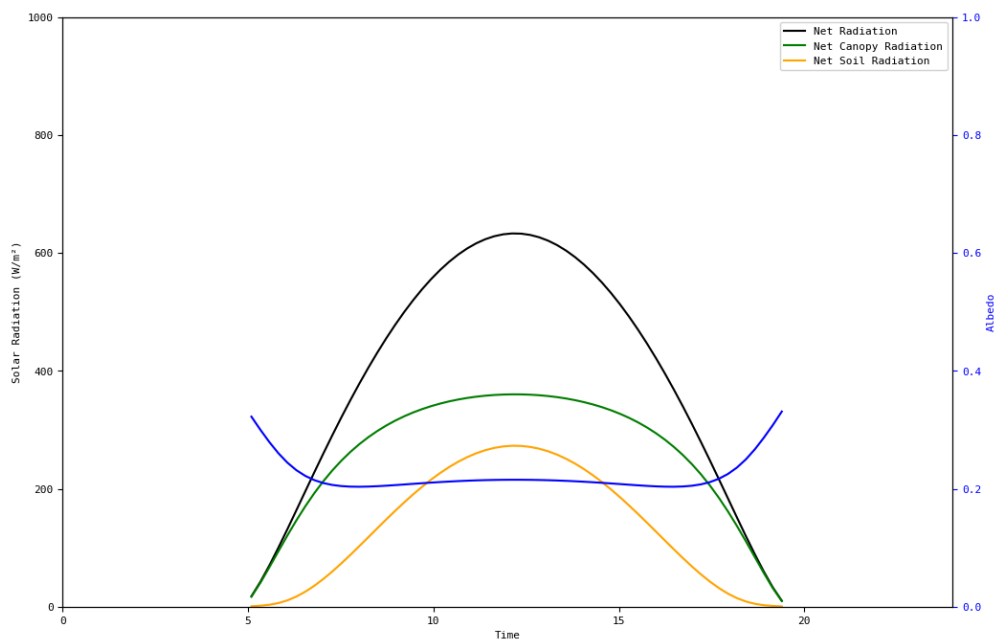
On the other hand, these simulations assume a constant ratio of diffuse irradiance along the day (fixed by the value set by sky_l). This is unrealistic in most cases, since around sunrise and sundown diffuse radiation tends to be larger, even under clear sky conditions. But for simplicity in these simulations, we made this assumption.

The plot depicts the net radiation at the land surface (S_n , black line) as well as the net radiation partitioning between crop ($S_{n,C}$, green line) and ground ($S_{n,S}$, yellow line). The secondary axis displays the albedo in blue line.

$$\begin{aligned}
 S_{n,C} + S_{n,S} &= S_n \\
 \alpha &= 1 - S_n / S_{down}
 \end{aligned}
 \tag{2.5}$$

```

w_sn = interactive(fn.plot_net_solar_radiation,
                  lai=fn.w_lai, leaf_angle=fn.w_leaf_angle, h_c=fixed(1),
                  f_c=fixed(1),
                  sdn_day=fn.w_sdn,
                  row_distance=fixed(1), row_direction=fixed(1),
                  skyl=fn.w_skyl,
                  fvis=fixed(0.55), lat=fn.w_lat, cab=fn.w_cab, cw=fn.w_cw,
                  soil_type=fn.w_soil)
display(w_sn)
  
```



```
interactive(children=(FloatSlider(value=2.0, continuous_update=False,
description='LAI (m2/m2)', max=4.0, tool...
```

Important

All parameters for all the plots used in this notebook are synchronized: any parameter change in one of the interactive plots is as well changed for the other plots, with all the graphs updated accordingly. This allows a quick intercomparison between graphs and simulations.

- While keeping the chlorophyll and water content constant, observe how net radiation barely changes with variations of LAI. However, the partitioning of net radiation between the canopy and soil does drastically change with LAI. This has an important implications since LAI is key for the radiation partitioning and, thus, for the evaporative/transpirative capacity of soil and vegetation, as well as for photosynthesis.
- Now keep relatively low values of LAI (<1.5) and inspect the effect of leaf angular distribution. More vertical leaves ($LIDF \rightarrow 90^\circ$) would provoke a reduction of intercepted, and thus absorbed, radiation near solar noon. Watch now that if diffuse radiation increases ($Skyl \rightarrow 1$), this effect is no longer evident. In this case, most of the radiation is coming equally from any direction of the hemisphere, making leaf inclination less relevant.
- Change the soil type and see how soil albedo affects net radiation, in particular at lower values of LAI when soil is more exposed to solar irradiance.

See Also

If you want to know more details about these calculations you can check the [pyTSEB Github source code](#).

2.7 Shortwave radiation in clumped canopies

So far, all simulations and processes have assumed a horizontally homogeneous canopy or crop, or at least dense enough to approximate a homogeneous crop. However, what happens when one is working with clumped canopies such as row crops or isolated canopies, e.g. in vineyards or orchards?

The hard fact is that everything is much more complex as on the one hand the canopy is usually clumped around a stem and branches, leaving some parts of the ground fully exposed. On the other hand, in row crops, the dimension of the canopy/trellis system and the orientation of the rows also affect how light is intercepted and transmitted between the crop and the ground.

In order to evaluate this situation, we need to consider additional parameters of crop structure:

- The canopy height (h_c), since higher canopies will cast longer shadows in the interrow.
- The fraction of canopy occupied by the ground (f_c), or the ratio between the canopy/trellis width (w_c) and the row spacing (L).
- The azimuth row orientation, with respect to the north. 0° will indicate that the rows are oriented S-N, negative values indicate that the rows go from NW to SE, and positive values from NE to SW, with -90° and/or 90° showing rows oriented E-W.

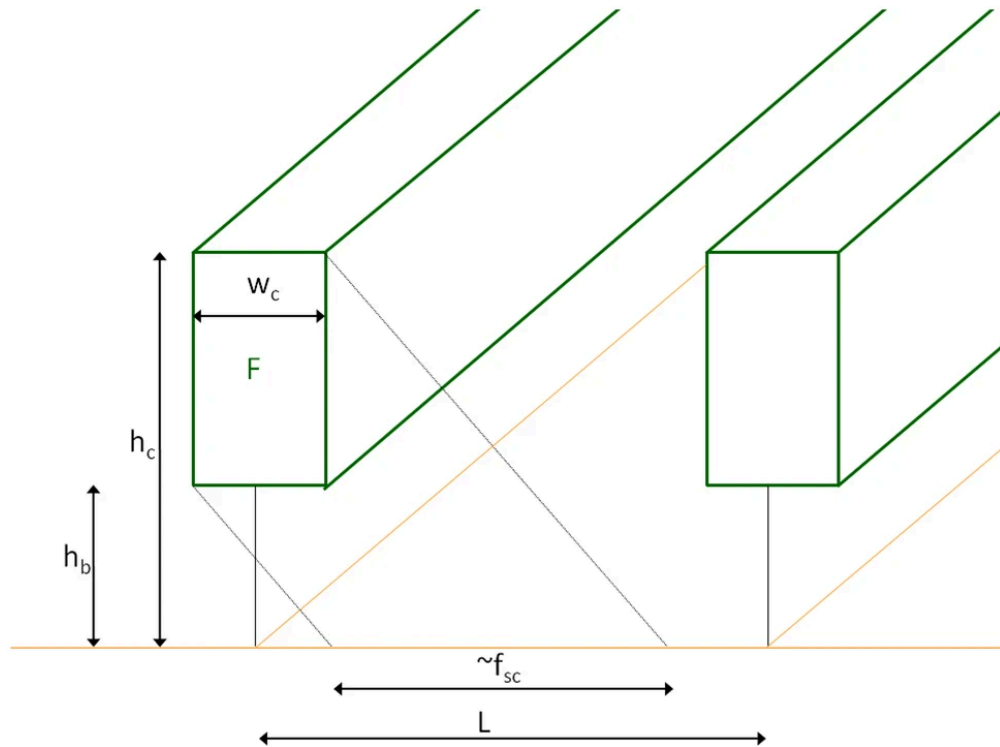


Figure 2.4: Canopy model for estimating the clumping index in row crops, F is the leaf area index, L is the distance between rows, hc is the canopy height, hb is the height of the canopy above ground, and wc is the canopy width

The approach builds upon the concept of the clumping index, defined as a conversion factor that modifies the leaf area index of a real canopy (F) in a fictitious homogeneous canopy with $LAI_{eff} = \Omega(\theta, \psi)F$, such as its gap fraction ($G(\theta, \psi)$) is the same as the gap fraction of the real-world canopy:

$$\Omega(\theta, \psi) F \kappa_b(\theta) = -\log[G(\theta, \psi)] \quad (2.6)$$

with $\kappa_b(\theta)$ is the beam extinction coefficient computed for a homogenous canopy (e.g. using Campbell & Norman (1998)).

The real canopy gap fraction is estimated on this simplified model as the sunlit part of the bare soil that is not shaded by the canopy ($1 - f_{sc}$) plus the gaps caused by the solar beam passing through the crop canopy (ignoring mutual shadowing between rows:

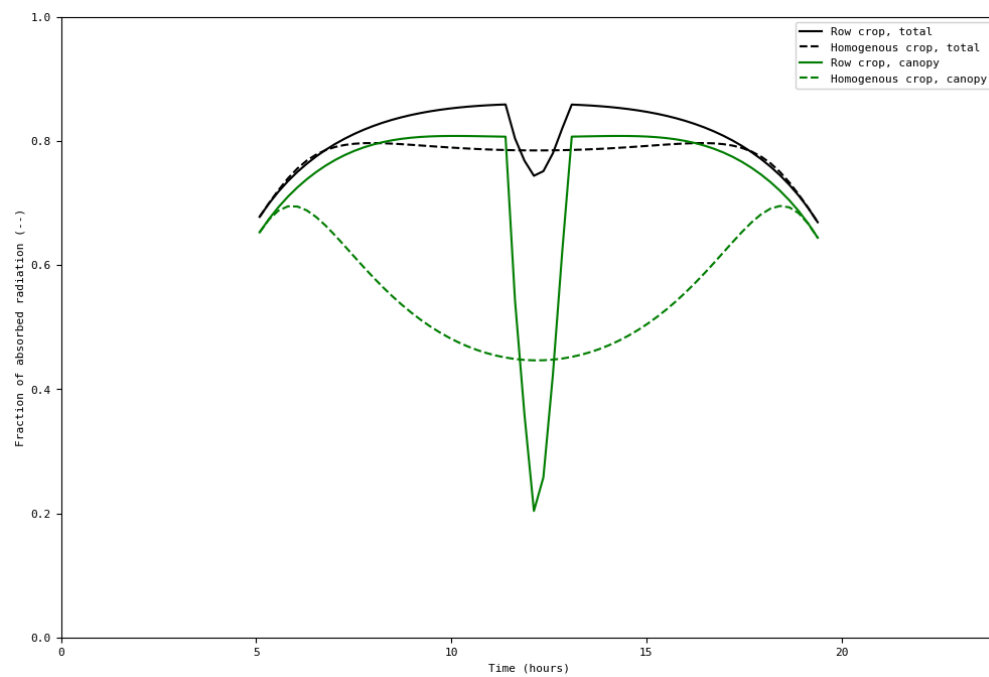
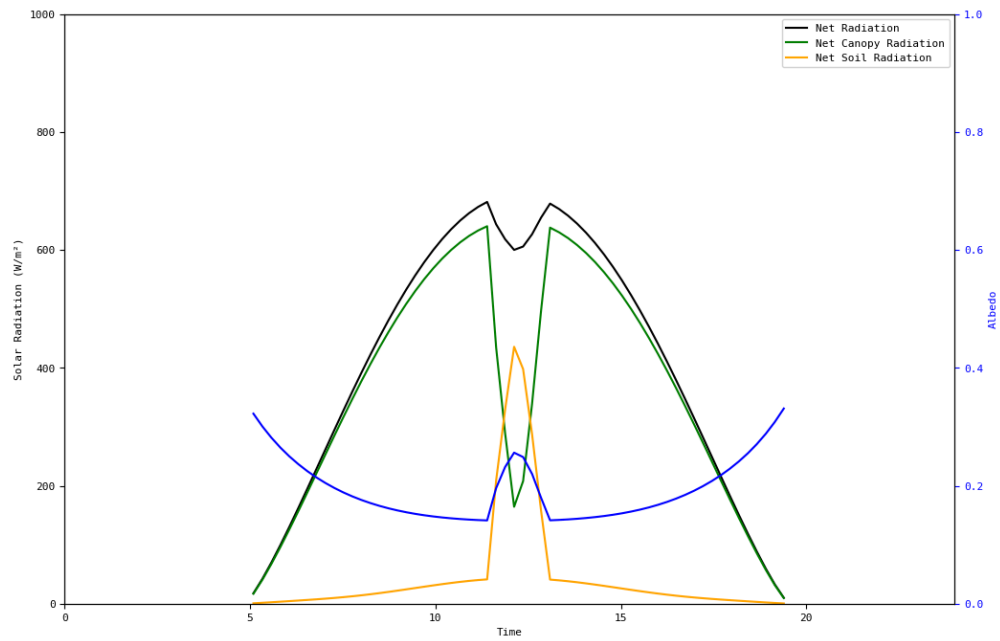
$$G(\theta, \psi) = f_{\text{sc}}(\theta, \psi) \exp[-\kappa_{\text{b}}(\theta)F] + [1 - f_{\text{sc}}(\theta, \phi)]. \quad (2.7)$$

and f_{sc} is computed from (#row-crop-model) using trigonometry:

$$f_{\text{sc}}(\theta, \phi) = \frac{w_{\text{c}} + (h_{\text{c}} - h_{\text{b}}) \tan \theta \mid \sin \phi \mid}{L} \quad (2.8)$$

In the following interactive plot, we have added these parameters to see their effect on net radiation. In addition, to ease the comparison with previous plot, we have included a second graph that shows the fraction of absorbed radiation for both a horizontally homogeneous crop and a row crop.

```
w_sn = interactive(fn.plot_net_solar_radiation,
                  lai=fn.w_lai, leaf_angle=fn.w_leaf_angle, h_c=fn.w_hc,
                  f_c=fn.w_fc,
                  row_distance=fn.w_interrow, row_direction=fn.w_psi,
                  sdn_day=fn.w_sdn, skyl=fn.w_skyl,
                  fvis=fixed(0.55), lat=fn.w_lat, cab=fn.w_cab, cw=fn.w_cw,
                  soil_type=fn.w_soil)
display(w_sn)
```



interactive(children=(FloatSlider(value=2.0, continuous_update=False, description='LAI (m²/m²)', max=4.0, tool...

- The sun rises from the eastern side, at noon reaches its zenith towards the north (or south depending on whether we are north or south of the equator), and sunset is from the western side. Taking into account these facts, observe the effect of changing the row orientation from E-W to S-N. There should be a drop in canopy net radiation when the sun is parallel to the rows, as in this case there should be lower radiation interception by the canopy.
- Observe that with larger canopy fractions (f_c , or wider canopies relative to the row spacing), the crop behaves similarly to the homogeneous crops from the previous simulation.
- There is an interaction effect between canopy height and row separation (L). Narrower rows in lower crops display a similar effect as that of taller crops in wider rows.
- Finally, as we have seen in previous interactive plots, these effects are minimized at larger proportion of diffuse radiation.

See Also

Check the code in the [pyTSEB GitHub repository](#).

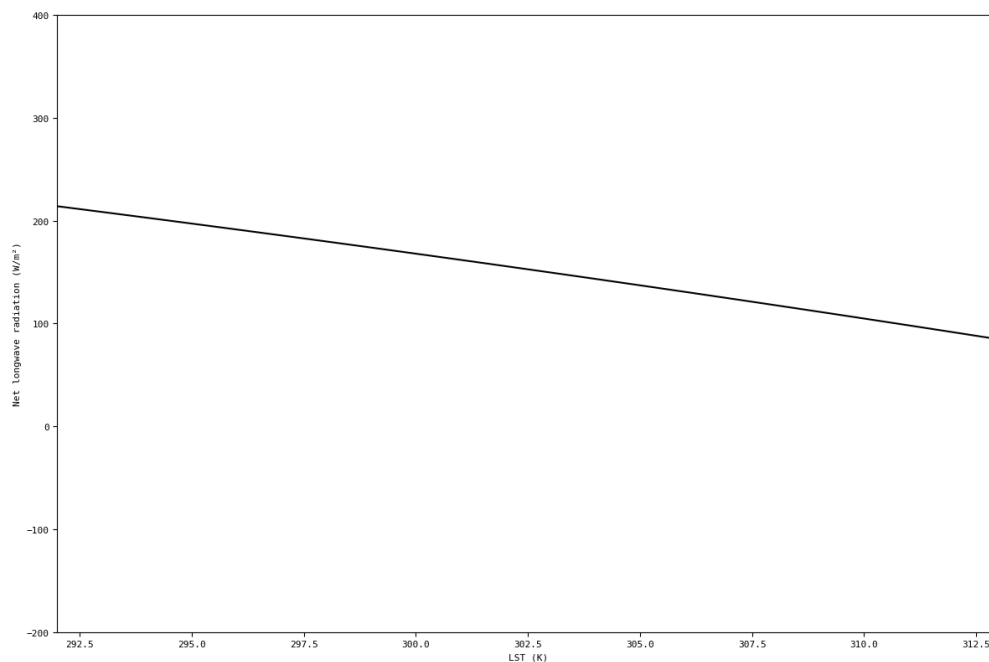
2.8 Net longwave radiation

Net longwave radiation (L_n) corresponds to the thermal emission of radiation from the land surface and the absorption of thermal irradiance from the atmosphere. L_n depends therefore on the surface's temperature and its emissivity, as well as on the atmospheric thermal emission (L_{down}), which at the same time depends on the atmospheric temperature and emissivity.

$$L_n = (1 - \varepsilon_{surf})L_{down} - \varepsilon_{surf}\sigma T_{surf}^4$$

In the next interactive plot, we will see how net longwave radiation changes as a function of atmospheric conditions (air temperature and humidity) and the surface's temperature and emissivity. Since surface temperature also depends on atmospheric conditions, we simulated various ranges of surface temperatures in the x-axis, from fully watered crops in which the surface temperature is closer to the air temperature, to stressed crops and bare/sparse areas, in which surface temperature is significantly hotter than the air.

```
w_ln = interactive(fn.plot_longwave_radiation, t_air=fn.w_tair, hr=fn.w_hr,
delta_t=fixed(np.linspace(-1, 20, 50)),
emiss=fn.w_emiss)
display(w_ln)
```



```
interactive(children=(FloatSlider(value=293.0, continuous_update=False,
description='T$_{air}$ (K)', max=313.0...
```

- Observe that the net longwave radiation yield values at a lower magnitude than the net shortwave radiation (especially for clear skies around solar noon time). Only in cases with high temperatures and humidity net radiation exceeds 400 W/m².
- Finally, the surface emissivity, the values of which usually range from 0.95 in arid areas with scarce vegetation to 0.99 in dense vegetation surfaces, has a lesser effect in the calculation of net longwave radiation. Therefore, for computing of net radiation its retrieval does not require very high accuracy.

2.9 Conclusions

In this exercise we have seen that net shortwave radiation depends mainly on:

1. Shortwave irradiance, which can be obtained from meteorological stations or from numerical weather models (e.g. [Copernicus Climate Data Store](#)).
2. Surface albedo

Furthermore, the surface albedo mainly depends on:

1. Leaf and soil spectral properties, mainly leaf chlorophyll content as the main pigment absorber in the PAR region.
2. Canopy structural characteristics, mainly LAI but also leaf angle distribution.
3. In addition, for row crops, the row architecture may be crucial in characterizing the radiation partitioning between ground and canopy.

The net longwave radiation has a lower contribution to the global net radiation (R_n), and its estimation is usually simpler, requiring:

1. Incident longwave irradiance, which can be obtained from meteorological stations or from numerical weather models (e.g. [Copernicus Climate Data Store](#)).
2. Surface emissivity and temperature, both of which can be obtained from thermal infrared Earth Observation data.

These analyses will allow you to better understand the radiative part of the energy balance. In addition, these simulations will permit evaluating the cost/benefit of using simpler or more sophisticated models for estimating net radiation and its partitioning between soil and canopy in structurally complex canopies.

In the next exercise, you will work on the other main factor of the energy balance that can be estimated with thermal infrared Earth Observation, the [sensible heat flux](#).

Note

Please feel free comment any thoughts.

3. Turbulent exchange of heat and water vapour

3.1 Summary

This interactive notebook has the objective of demonstrating the relationships between canopy structure, turbulent transport of heat and water and surface temperature.

For this purpose, we will use simulations based on energy combination models, both the “big leaf” (2013) model as well the dual source Shuttleworth & Wallace (1985) model.

3.2 Instructions

Read carefully all the text and follow the instructions.

Once each section is read, run the jupyter code cell underneath (marked as In []) by clicking the icon Run, or pressing the keys SHIFT+ENTER of your keyboard. A graphical interface will then display, which allows you to interact with and perform the assigned tasks.

To start, please run the following cell to import all the packages required for this notebook. Once you run the cell below, an acknowledgement message, stating all libraries were correctly imported, should be printed on screen.

```
%matplotlib inline
from ipywidgets import interactive, fixed
from IPython.display import display
import functions.tseb_and_resistances as fn

scikit-learn-intelex not available, skipping the use of Intel acceleration.
Try to consider installing Intel acceleration via `pip install scikit-learn-intelex`

Thanks! libraries imported
You can now continue with the following tasks
```

3.3 Basic equations

As a reminder, the estimation of evapotranspiration (ET), or latent heat flux (λE) in terms of energy, can be obtained from the energy balance equation:

$$\lambda E \approx R_n - G - H \quad (3.1)$$

In the previous exercise, we have already seen the theory and processes related to net radiation (R_n) and soil heat flux (G). Now, we will focus on the sensible heat flux (H) to finally get λE or ET.

The effective transport of sensible heat can be computed as:

$$H = \rho_{air} C_p \frac{T_0 - T_a}{r_a} \quad (3.2)$$

- ρ_{air} and C_p are the density of air and its specific heat respectively, and they can be assumed to be practically constant.
- T_0 (K) is the aerodynamic temperature.

Attention

Do not confuse this temperature with the radiometric surface temperature that is estimated with thermal infrared sensors

- T_a (K) is the air temperature measured at a height above ground z_T , and significantly above the canopy height (h_c)
- r_a is the aerodynamic resistance to heat transport (s/m), a reciprocal of which is the aerodynamic conductance.

Attention

Do not confuse this aerodynamic conductance with the leaf stomatal conductance

This fundamental equation can be interpreted in a way that the heat exchange between the land surface and the atmosphere (H) is driven by a gradient of temperatures between the surface and the nearest layers of air above the canopy. This transport or exchange of heat is modulated by the aerodynamic resistance, which includes a series of complex mechanical and convective processes that enhance or impede the transport of heat between the surface and the air:

$$r_a = \frac{\left[\ln\left(\frac{z_T - d}{z_{0H}}\right) + \Psi_H \right] \left[\ln\left(\frac{z_U - d}{z_{0M}}\right) + \Psi'_M \right]}{k^2 U} \quad (3.3)$$

- U is the wind speed (m/s) measured at a height above ground z_U , and significantly above the canopy height (h_c).
- z_T is the height (m) at which air temperature is measured or modelled.
- z_U is the height (m) at which wind speed is measured or modelled.
- d and z_{0M} are the zero-plane displacement height and surface roughness length for momentum transport, respectively. Usually these two variables can be estimated using a simplified formula based on canopy/obstacle height (h_c)

$$\begin{aligned} z_{0M} &= 0.125h_c \\ d_0 &= 2/3h_c \end{aligned} \quad (3.4)$$

- z_{0H} is the roughness length for heat transport. Likewise, its estimation is usually computed as a fraction of z_{0M} (e.g.

$$z_{0H} = 0.1z_{0M} \quad (3.5)$$

for dense vegetation)

- $k = 0.41$ is von K  rman constant.
- Ψ'_H and Ψ'_M are semi-empirical functions that modulate the aerodynamic resistance and the vertical wind attenuation as function of atmospheric stability.

See Also

If you want to know more details about their calculation, you can check the [pyTSEB](#) [GitHub source code](#) for Ψ'_H and for Ψ'_M .

From the equation above we can arithmetically deduce that the aerodynamic resistance to heat and vapour transport decreases with wind speed and with the surface aerodynamic roughness.

In the following tasks, we are going to disentangle all those processes step by step, allowing us to better understand and estimate r_a .

3.4 The wind profile

The wind speed profile with respect to the height above the ground can be estimated according to a logarithmic function. Ignoring in this first step the effects of convective turbulence, and thus considering a neutral atmospheric stability:

$$u(z) = \frac{u_*}{k} \left[\log\left(\frac{z - d}{z_{0M}}\right) \right] \quad (3.6)$$

Note

This logarithmic wind attenuation is closely related to the aerodynamic resistance.

See Also

You can also check the [pyTSEB GitHub source code](#).

The wind attenuation will be stronger for a larger surface roughness, expressed by the variables z_{0M} and d_0 .

u_* is the friction velocity. Likewise, simplifying and neglecting the effects of atmospheric stability, u_* is computed as:

$$u_* = \frac{k U}{\log\left(\frac{z_u - d_0}{z_{0M}}\right)} \quad (3.7)$$

See Also

You can also check the [pyTSEB GitHub source code](#).

Therefore, based on wind speed measurement at a given height, we could extrapolate the wind profile to estimate the wind speed right above the canopy (h_c):

$$u_C = \frac{u_*}{k} \left[\log\left(\frac{h_c - d}{z_{0M}}\right) \right] \quad (3.8)$$

3.4.1 The wind profile within the canopy

As the wind passes through the canopy, it is mainly attenuated by the presence of foliage. This attenuation within the canopy and towards the ground can have implications on the heat and vapour exchange between the canopy and the ground.

Usually this attenuation is formulated assuming that the canopy is horizontally and vertically homogeneous. However most canopies, and in particular woody vegetation, show a foliar density that varies with height: plants usually have a stem from which branches diverge, starting at a certain height above ground, resulting in canopies with a more or less irregular shape.

In the following interactive plot, we are going to simulate the wind attenuation above and below the canopy, and how this profile diverges according to the canopy structure. We are going to compare a profile for both a vertically heterogeneous (based on Massman et al. (2017) mode) and homogeneous canopy (based on [Goudriaan, 1977](#) model). In addition, we are going to see the standard wind profile as estimated in the reference ET method proposed by FAO. For this task, we are keeping a constant wind speed ($U = 5$ m/s) measured 10m above ground ($z_U = 10$ m), and assuming a neutral atmospheric stability.

See Also

You can check the GitHub source code for both the [Goudriaan](#) and for the [Massman](#) models

In order to characterize our canopy, we are going to need:

- Leaf Area Index (LAI) and canopy height (h_c), these two parameters are common for the homogeneous and heterogeneous canopy.
- The height above ground in which the foliage begins (h_{bottom}), expressed as the ratio to the canopy height

Note

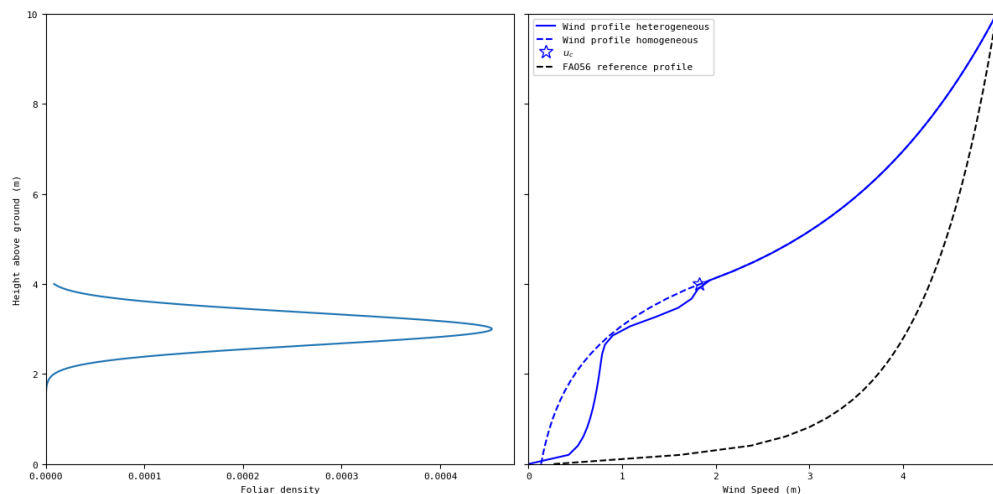
By default this value is set at 0.5, meaning that the foliage starts at a height of $0.5h_c$ m.

- The height at which the maximum foliage density is located (h_{max}), expressed as a ratio of the canopy length (i.e the difference between h_c and h_{bottom}).

In the graph below, the left-panel plot shows the relative foliage density of the simulated canopy.

The right-panel plot shows the wind speed attenuation from 10m to the ground (0m) for both the vertical heterogeneous (solid blue line) and homogeneous (dashed blue line) cases. A blue star indicates the wind speed right above the canopy, and the dashed black line shows the standard profile for the reference ET crop in FAO method (a homogeneous grass layer with $h_c = 0.12$ m and $LAI \approx 3$).

```
w_wind = interactive(fn.wind_profile_heterogeneous, lai=fn.w_lai,
zol=fixed(0.), h_c=fn.w_hc,
                    hb_ratio=fn.w_hb_ratio, h_c_max=fn.w_h_c_max)
display(w_wind)
```



```
interactive(children=(FloatSlider(value=2.0, continuous_update=False,
description='LAI (m2/m2)', max=4.0, tool...
```

- See how different the wind profile can be compared to the FAO reference crop profile. This is particularly evident when h_c is significantly larger than 0.12 m. In addition, with higher canopies, there will be clearer differences between a profile for a vertically homogenous crop compared to a heterogenous one.
- As a rule of thumb, the stronger the attenuation is, the more turbulent the exchange of heat and vapour will be. This turbulence will produce more eddies and a more efficient transport of heat and vapour between the soil, the plant and the atmosphere, since eddies move the parcels of (warm and humid) air in contact with the land surface towards the atmosphere.

See Also

Figure 3.1: This is a YouTube video showing a simulation of eddies while surface roughness is increased. In this case it corresponds to an airfoil

- See also that with higher leaf density (expressed in terms of LAI), the wind attenuation within the canopy will be stronger. This results in distinct heat and water exchange between the canopy and the ground.
- Finally, see that the strongest attenuation within the canopy occurs around the height of maximum foliage density. This is due to the fact that the wind is encountering a larger amount of obstacles (leaves) and, thus, loses kinetic energy and, hence, speed. On the contrary, in areas closer to the ground, where the foliage density is minimal, the wind is less attenuated.

3.4.2 Effect on the aerodynamic resistance

In this task, you will explore how the aerodynamic resistance varies with surface roughness (characterized by canopy height), wind speed and atmospheric stability.

3.4.2.1 The atmospheric stability and convective turbulence

So far, we saw that arithmetically the aerodynamic resistance to heat and vapour transport decreases with wind speed and roughness. In other words, the exchange of heat and vapour between the surface and the atmosphere becomes more efficient with stronger winds and rougher surfaces.

Now, we are going to evaluate that with more unstable atmospheres, due to convection, the turbulence is enhanced and, thus, the heat exchange will be more efficient.

The atmospheric stability can be expressed in relative terms as a ratio between aerodynamic roughness length (z_{0m}) and the Monin-Obukhov length ($\xi = z_{0m}/L$), the latter of which also depends on the sensible heat flux (H).

See Also

You can also check the [pyTSEB GitHub source code for the Monin-Obukhov length](#)

The more negative this ξ coefficient is, the more unstable the atmosphere is and, thus, the more efficient the transport of heat and vapour would be due to the convection of warm air upwards the cooler atmosphere.

Note

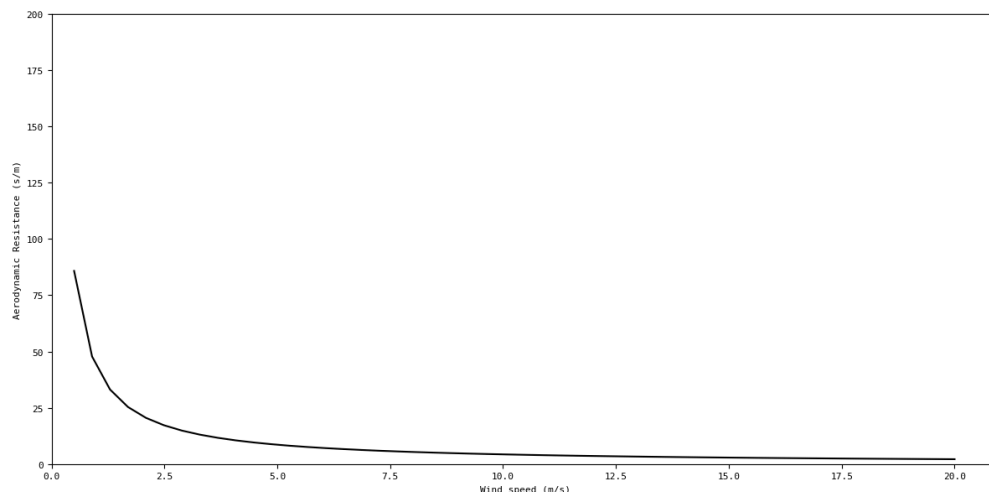
As a reminder, the density of warm air is less than that of cool air, resulting in a vertical (and turbulent) transport of air upwards. [See a video of this phenomenon](#)

Important

At daytime, the land surface is absorbing heat (mainly due to sun irradiance) and, therefore, unstable conditions usually occur ($\xi < 0$) during the daytime. By contrast, at nighttime, the land surface can be cooler than the air and, thus, thermal inversion phenomena and atmospheric stability can occur ($\xi > 0$).

The next interactive plot will let you evaluate the joint effect of atmospheric stability, wind speed and canopy height/roughness on the characterization of aerodynamic resistance.

```
w_ra = interactive(fn.plot_aerodynamic_resistance, zol=fn.w_zol, h_c=fn.w_hc)
display(w_ra)
```



```
interactive(children=(FloatSlider(value=0.0, continuous_update=False,  
description='$\\xi$', max=1.0, min=-1.0,...
```

Note

All parameters for all the plots used in this notebook are synchronized: any parameter change in one of the interactive plots is as well changed for the other plots, with all the graphs updated accordingly. This allows a quick intercomparison between graphs and simulations.

- See how the aerodynamic resistance reduces exponentially with wind speed, but also with surface roughness/canopy height. This issue can have crucial implications when interpreting the gradient of temperature between the surface and the air.

Warning

The same temperature gradient can indicate different levels, or lack thereof, of water stress, depending on the surface roughness and the actual meteorological conditions.

- Now change the stability coefficient and see how r_a changes. The aerodynamic resistance is reduced with more negative xi values for a given wind speed and surface roughness. By contrast, with stable conditions ($xi > 0$) the vertical transport of heat and vapour is reduced (larger values of r_a).

3.5 The evapotranspiration and the sensible heat flux

In this second part of the exercise, we are going to integrate everything we have learnt so far to evaluate their joint effect on evapotranspiration and the land surface temperature. As mentioned before, we will use two popular energy combination models to perform our simulations.

The main advantage of (2013) model is its relative simplicity (this is the model used to estimate FAO's reference ET), while its main disadvantage is that it is not fully able to partition fluxes between soil and canopy (e.g. separation of soil evaporation and canopy transpiration). This issue has important implications when modelling ET over sparse vegetation or row crops.

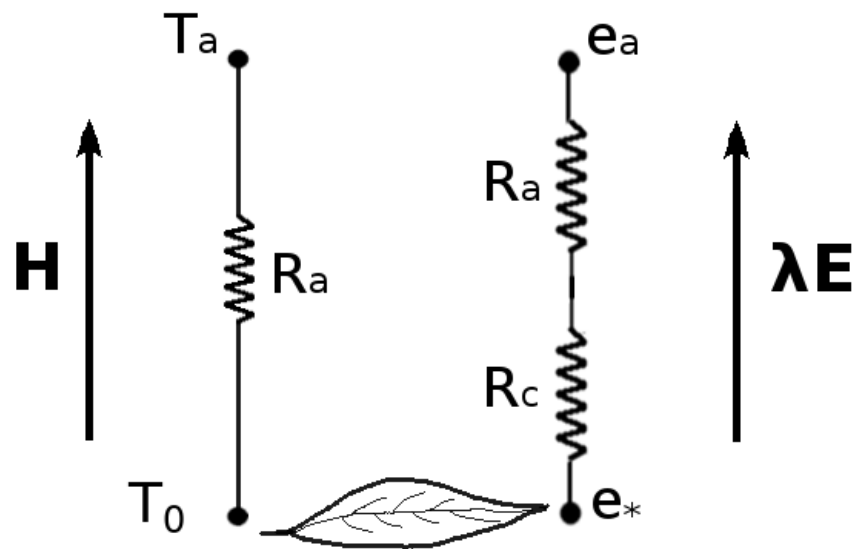


Figure 3.2: Resistances layout in the (2013)

Note

The most well known application of Penman-Monteith model is the calculation of reference ET, e.g. [FAO 56](#) or (2005), which estimates the evapotranspiration for a well watered crop used as reference. This reference crop for FAO 56 is defined as a perfectly homogeneous grass layer clipped at 12cm height.

On the other hand, the Shuttleworth & Wallace (1985) model is more adapted to sparse vegetation conditions and is able to partition fluxes between soil and canopy, allowing thus the separation between soil evaporation and canopy transpiration. However, this model is more complex than Penman-Monteith and, thus, requires additional inputs.

Additional variables which are required by the Shuttleworth-Wallace model include the soil resistance of vapour transport (R_{ss}), which is related to the topsoil moisture content.

R_{ss} values closer to 0 would indicate topsoil saturated or even flooded with water, whereas larger values will indicate a drier soil surface.

The leaf stomatal conductance or its reciprocal the leaf stomatal resistance ($g_s = 1/r_{st}$) is an indicator of crop water stress. g_s usually shows values around a $0.4\text{--}0.5 \text{ mmol m}^{-2} \text{ s}^{-1}$ when the crop is well watered.

According to FAO guidelines, the stomatal resistance of the reference well-watered crop is 100 s/m, which is equivalent to a stomatal conductance of 0.415 mmol m⁻² s⁻¹ (at standard atmospheric conditions of 25°C at sea level).

Any exchange of heat and vapour between the ground, vegetation and the atmosphere is produced under a turbulent transport. Therefore, if the ground surface is dry and receives enough radiation it will heat up and, thus, will enhance the transport of dry and hot air towards the canopy. This would enhance the evaporative demand of the canopy (warming and drying the air around the leaves). Inversely, a dense and well irrigated canopy can

provide cool and moist air towards the ground environment, decreasing its evaporative demand.

The following interactive plot will run simulations of (2013) and Shuttleworth & Wallace (1985) models for a range of LAIs, given the following standard meteorological forcing:

- Solar irradiance of 300 W m^{-2} .
- Air temperature of 25°C (298K).
- 50% relative humidity, which is equivalent to a vapour pressure deficit of $VPD \approx 15\text{mb}$.
- Wind speed of 5 m s^{-1} measured at 10m above ground.

... and fixed surface albedo of 0.23 as defined for FAO's reference ET.

See Also

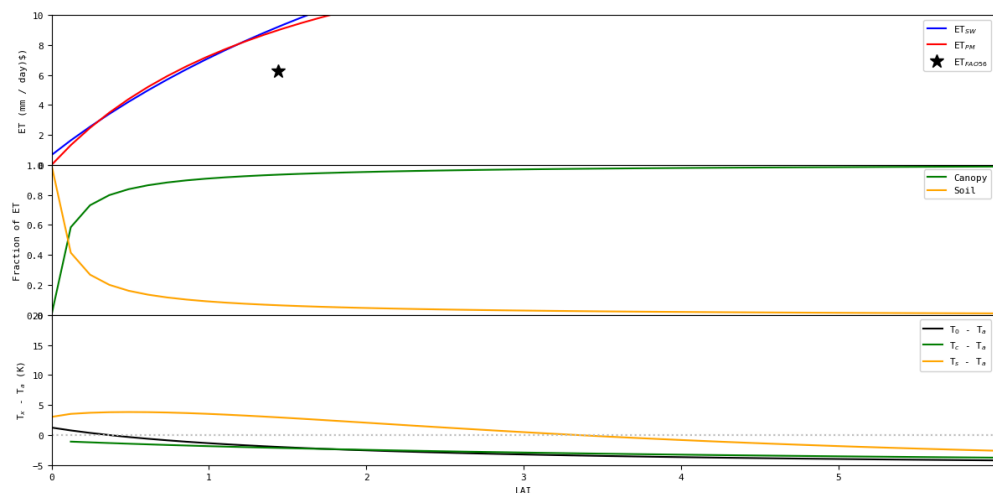
You can check the PyTSEB GitHub source codes for the [Penman-Monteith model](#), for the [Shuttleworth-Wallace model](#), the [FAO56 reference ET](#), and the [ASCE reference ET](#)

The upper panel of the plot shows the simulated daily ET for the Shuttleworth & Wallace (1985) (blue line) and (2013) (red line) models. In addition, it shows the FAO reference ET value with a black star.

The middle panel shows the ET partitioning between soil evaporation (yellow line) and canopy transpiration (green line) as simulated by Shuttleworth-Wallace.

The lower panel shows the Shuttleworth & Wallace (1985) simulated soil (yellow line), canopy (green) and aerodynamic temperatures (black line), expressed as their respective differences from the air temperature.

```
w_et = interactive(fn.fluxes_and_resistances, r_ss=fn.w_r_ss, g_st=fn.w_g_st,
h_c=fn.w_hc)
display(w_et)
```



```
interactive(children=(FloatSlider(value=0.415, continuous_update=False,
description='g_{st}$ (mmol/m^2 s^{-1})', ma...
```

- Observe how the graphs change with R_{ss} (e.g. the topsoil moisture). See how the total ET in Shuttleworth-Wallace changes as opposed to the Penman-Monteith that can not directly consider the topsoil moisture. Observe how ET increases with LAI for the given parameters, since surface resistance decreases with LAI ($R_c = r_{st}/LAI$).

Warning

Remember that R_{ss} represents the topsoil moisture condition. Indeed, it is possible (more often than one could expect) to have a dry soil surface ($R_{ss} \gg 1000 \text{ s m}^{-1}$) while having a well-watered canopy ($g_{st} \gg 0 \text{ mmol m}^{-2} \text{ s}^{-1}$), since plants extract water deeper in the soil, where water is still stored.

- See in the middle panel how transpiration dominates total ET at larger LAI values. In the previous exercise, we showed that, at larger LAI values, the canopy will intercept and absorb more energy, and thus will have a larger evaporative capacity.
- Look now at the ET values for $LAI \rightarrow 0$ when $R_{ss} \rightarrow 0$. Penman-Monteith, since it is a “big-leaf” model and does not directly incorporate the transport of water from the soil surface, is not able to simulate the soil surface evaporation. On the other hand, with R_{ss} values above 5000 s/m both models yield very similar values of ET.
- Observe in the lower panel how temperatures decrease with LAI: a dense canopy dissipates the heat more efficiently than a less dense canopy, even under the same water stress conditions.
- Check also how soil temperature increases with R_{ss} .

Tip

You could even perceive how the leaf temperature changes slightly with R_{ss} , especially at lower LAI values. This is related to the flux interaction between ground and canopy that we mentioned before: a dry and warm (moist and cool) soil transports air towards the canopy, which can increase (decrease) the evaporative demand and, thus, decrease (increase) the canopy temperature.

- Keep a value of $R_{ss} = 5000$ fixed (dry soil surface) and check now how ET changes with decreasing values of stomatal conductance, and thus simulating more severe crop water stress conditions. The ET for both Penman-Monteith and Shuttleworth-Wallace decreases and deviates further from FAO’s reference ET value (optimal water conditions). In addition, the more stressed the plant (i.e. at lower g_{st} values), the lower the transpiration, with no transpiration for $g_s = 0 \text{ mmol m}^{-2} \text{ s}^{-1}$. Finally, observe that the lower g_{st} , the hotter the leaf and aerodynamic temperatures.

Tip

You could even perceive how the soil temperature slightly changes with g_{st} , especially at larger LAI values. This is again related to the flux interaction between ground and canopy that we mentioned before: a dry and warm (moist and cool) canopy transports air towards the ground, which can increase (decrease) the evaporative demand and thus decrease (increase) the soil temperature.

- See now the effect of canopy height/roughness. Restore the values of stomatal conductance back to $g_s = 0.41 \text{ mmol m}^{-2} \text{ s}^{-1}$ and $R_{ss} = 5000$. Increase now the canopy height, from an initial value of 0.12m, and observe how ET and temperature changes with canopy roughness for a given moisture status.

3.6 About the surface temperature

We already mentioned that the aerodynamic temperature (T_0), which is the temperature that holds valid the sensible heat flux equation, is different to the radiometric surface temperature (LST) estimated with Earth observation remote sensing. It is therefore important to be aware of such difference and to be able to correctly interpret LST when estimating sensible heat flux and/or ET.

Note

Indeed, the different remote sensing ET models mainly differ on how they use the LST in order to retrieve H and/or λE .

Firstly, since heat is exchanged less efficiently than momentum, we need to correctly define the aerodynamic roughness length for heat transport (z_0H), and, indeed, the relationship $z_0H \approx 0.1z_{0M}$ might not be valid for all types of surfaces.

Secondly, the radiometric surface temperature can significantly change depending on the observation viewing angle. This is particularly relevant for Earth observation sensors with a wide field of view or wide swath, such as Sentinel-3 SLSTR, with view zenith angles ranging from nadir ($VZA = 0^\circ$) up to VZAs of 50° or 60° . This is important since the sensor might see a different proportion of (cool) vegetation and (warm) soil. Indeed, we saw in the previous exercise that the canopy transmittance depends on the canopy density, the leaf angle distribution, and the incidence/illumination angle. This effect is perfectly reciprocal when we deal with the sensor geometry instead of the solar geometry.

The interactive plot below demonstrates this effect. You can define the sensor's view zenith angle as well as the dominant leaf inclination angle. According to the soil and canopy temperatures estimated in the [previous simulation](#), and after setting as well the soil and leaf emissivity, the graph will display the estimated LST for a range of LAIs.

Tip

You can modify the simulation parameters [here](#) and the changes will be reflected in the simulated LST.

The plot shows two different methods of estimating the radiometric land surface temperature:

- A detailed calculation based on the 4SAIL model that includes thermal emission and multiple thermal scattering between soil and vegetation.

See Also

You can check the [Python version of 4SAIL model](#)

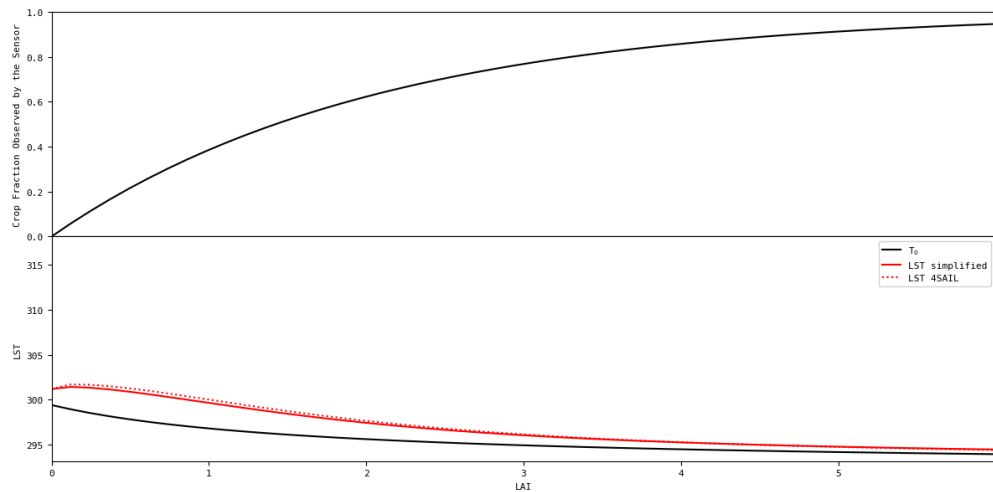
- A simplified method which considers a weighted average based on the fraction of vegetation observed by the sensor, which is the one implemented in TSEB:

$$LST^4 = f_c(\theta)T_C^4 + [1 - f_c(\theta)]T_S^4 \quad (3.9)$$

See Also

You can check the [pyTSEB GitHub source code](#)

```
w_fveg = interactive(fn.get_land_surface_temperature,
                    vza=fn.w_vza, leaf_angle=fn.w_leaf_angle,
                    temperatures=fn.w_et,
                    e_v=fn.w_ev, e_s=fn.w_es)
display(w_fveg)
```



```
interactive(children=(FloatSlider(value=0.0, continuous_update=False,
description='VZA (deg.)', max=89.0, step...
```

- Notice that with increasing VZA, the LST is decreasing, with this decrease being more significant at higher VZAs ($VZA > 35^\circ$) and for intermediate values of LAI.
- There might be significant differences between the aerodynamic temperature and the LST, in particular for less developed and sparse canopies ($LAI < 2$).

3.7 Conclusions

In this exercise, we have seen that exchange of heat and vapour between ground, vegetation and the atmosphere depend on:

1. The topsoil moisture content, which drives the soil evaporation.
2. The root-zone soil moisture content, which drives the canopy transpiration and its photosynthetic capacity.
3. The canopy structure, which drives how efficiently the canopy dissipates the heat and vapour towards the atmosphere.

The radiometric land surface temperature is a key factor in order to evaluate the crop water stress and crop evapotranspiration.

Note

Please feel free comment any thoughts.

4. How TSEB and 3SEB works?

4.1 Summary

This interactive Jupyter Notebook has the objective of showing the implementation of the TSEB-PT and 3SEB-PT models using the [pyTSEB package](#) and [py3SEB package](#).

4.2 Instructions

Read carefully all the text and follow the instructions.

Once each section is read, run the jupyter code cell underneath (marked as In []) by clicking the icon Run, or pressing the keys SHIFT+ENTER of your keyboard. A graphical interface will then display, which allows you to interact with and perform the assigned tasks.

To start, please run the following cell to import all the packages required for this notebook. Once you run the cell below, an acknowledgement message, stating all libraries were correctly imported, should be printed on screen.

```
%matplotlib inline
from pathlib import Path
from ipywidgets import interact, interactive, fixed, widgets
from IPython.display import display
import numpy as np
from pyTSEB import TSEB
from py3seb import py3seb
import matplotlib.pyplot as plt
import pandas as pd
from plotly.subplots import make_subplots
import plotly.graph_objects as go
print("Libraries imported!")
```

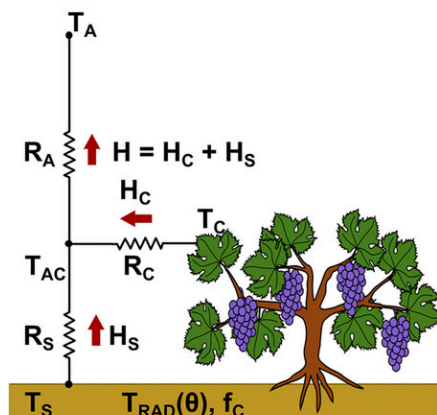
Libraries imported!

4.3 The TSEB model

The Two-Source Energy Balance (TSEB) model Norman et al. (1995) was specifically designed to account for directionality of radiometric temperature measurements and to avoid the use of any empirical “excess” resistance terms in the model. To achieve this, both the heat and water fluxes are separated into a soil and canopy layer, with a set of resistances set in series.

Note

In Norman et al. (1995) the original resistance scheme was in parallel



Sensible heat flux module

System, soil (S), canopy (C) budgets

$$RN = H + LE + G$$

$$RN_S = H_S + LE_S + G$$

$$RN_C = H_C + LE_C$$

Two-source approximation

$$T_{RAD}(\theta)^4 \approx f_C(\theta)T_C^4 + (1 - f_C(\theta))T_S^4$$

Temperature constraint

$$H_C, H_S, RN_C, RN_S, G$$

PT, PM, or LUE R_C model

$$LE_C$$

Residual

$$LE_S = RN - H - G - LE_C$$

Iterative energy balance solution

Figure 4.1: Schematic diagram of the TSEB model resistance network for sensible heat flux and the basic set of equations used to obtain an iterative solution. Source: Kustas et al. (2018).

Energy fluxes are therefore split into soil and canopy, considering the conservation of energy:

$$\begin{aligned} R_n &\approx H + \lambda E + G \\ R_{n,S} &\approx H_S + \lambda E_S + G \\ R_{n,C} &\approx H_C + \lambda E_C \end{aligned} \quad (4.1)$$

with R_n being the net radiation, H the sensible heat flux, λE the latent heat flux or evapotranspiration, and G the soil heat flux (all fluxes are expressed in W m^{-2}).

See Also

An additional notebook is dedicated to the calculation of Net Radiation [here](#)

The approximation in Eq. (1) reflects additional components of the energy balance that are usually neglected, such as heat advection, storage of energy in the canopy layer or energy for the fixation of CO_2 , which are not computed by the model.

Remote sensing Energy Balance Models, REBMs, rely on the ability of the radiometric information to estimate, net radiation, soil heat flux and sensible heat flux Norman et al. (1995). To see how pyTSEB works we will run several pieces of code to estimate each of the components of the energy balance.

See Also

This is a full description of the EC column fields:

[./fluxnet2015_variables.md](#)

4.3.1 Sensible heat flux

The key in TSEB models is the partition of sensible heat flux into the soil and canopy layers, which depends on the soil and canopy temperatures (T_S and T_C respectively, left side of Fig. [\#tseb](#)). Given the difficulty of obtaining the pure component temperatures, even with very high resolution data due to canopy gaps, Norman et al. (1995) found a solution to retrieve T_S and T_C using a single observation of the directional radiometric temperature $T_{rad}(\theta)$ as this is the case for most of the remote sensing systems. Eq. (2) decomposes the composite $T_{rad}(\theta)$ temperature between its components T_S and T_C , assuming blackbody emission of thermal radiance:

$$\sigma T_{rad}^4(\theta) = f_c(\theta) \sigma T_C^4 + [1 - f_c(\theta)] \sigma T_S^4 \quad (4.2)$$

where $f_c(\phi)$ is the fraction of vegetation observed by the sensor. Since (2) consists of two unknowns and only one equation, an iterative process to find H_S , T_S , H_C and T_C is defined based upon an initial guess of potential canopy transpiration, and under the assumption that during daytime hours condensation for the soil/substrate should not occur. The initial canopy latent and sensible heat fluxes are estimated based on the PRIESTLEY & TAYLOR (1972) formulation for potential transpiration (Eq. (3)) and Eq. (1).

$$\begin{aligned} \lambda E_C &= \alpha_{PT} f_g \frac{\Delta}{\Delta + \gamma} R_{n,C} \\ H_C &= R_{n,C} - \lambda E_C \end{aligned} \quad (4.3)$$

where α_{PT} is the Priestley-Taylor coefficient, initially set to 1.26, f_g is the fraction of vegetation that is green and hence capable of transpiring, Δ is the slope of the saturation vapour pressure versus temperature, γ is the psychrometric constant. T_C is then computed

by inverting the equation for turbulent transport of heat between the surface and the reference height above the surface)

With a first estimate of T_C , soil temperature is computed from Eq. (2) and then soil sensible and latent heat fluxes. At this stage, if soil latent heat flux results to be non-negative, a solution is found, otherwise the Priestley-Taylor coefficient is reduced incrementally, reducing hence the canopy transpiration, in order to avoid negative soil latent heat flux, until a realistic solution is found (no condensation occurring neither in the soil nor in the canopy in daytime).

See Also

For more details the reader is addressed to the works of Norman et al. (1995) or Kustas & Norman (1999)

The full code of TSEB-PT is implemented in the pyTSEB GitHub repository ([pyTSEB/TSEB.py](#)). The general workflow is described here:

Note

There are two main steps in TSEB-PT:

- A first step that prepares all ancillary variables, such as air density, heat capacity of air..., and initializes a priori values of T_C , T_S , Ln , and G : From line 728 to line 756.
- A second iterative step that consists on two nested loops:
 - an outer loop that searches for convergence in the friction velocity and Monin-Obukhov length and serves to recalculate the aerodynamic resistances: From line 756 to line 903
 - an inner loop that starts with the potential Priestley-Taylor coefficient and iteratively starts being reduced for each case individually until non-negative turbulent fluxes are obtained for both canopy and soil: From line 780 to line 882

Note

The outer loop regarding the recalculation of atmospheric stability factors tends to converge pretty fast under daytime/unstable conditions, but on the other hand it has difficulties in converging under stable/nighttime conditions. Therefore in pyTSEB it is set a number of maximum iterations for this loop reflected in the [ITERATIONS](#) constant

4.3.2 Soil heat flux

Soil heat flux is usually estimated as a fraction of the soil net radiation $R_{n,S}$. Such fraction can be assumed constant along the day (e.g. $G = 0.35R_{n,S}$) or can change along the day, such as the sinusoidal function proposed by [Santanello and Friedl, 2003](#).

Since the computation of net longwave radiation is dependant of getting estimates of T_C and T_S and thus is computed internally in the TSEB modules during the iteration that will be discussed in the next section, G is also computed within the TSEB function, being iteratively recomputed every time a new T_C and T_S is obtained. The different TSEB versions in pyTSEB accept computing G in four different ways:

1. As a constant ratio of $R_{n,S}$ [function `calc\_G\_ratio`](#)
2. As time dependent ratio of $R_{n,S}$ according to a sinusoidal function [function `calc\_G\_time\_diff`](#)
3. As time dependent ratio of $R_{n,S}$ according to a double assymetric sigmoid function [pyTSEB/TSEB.py](#)
4. As precomputed/prescribed values

Note

This last option can be chosen to force the use of measured G when running TSEB, which sometimes can be useful when testing and evaluating new (sub)modules into TSEB.

4.3.3 Surface roughness

In addition, REBMs need additional ancillary inputs, such as air temperature, wind speed and canopy height or roughness. The latter accounts for the efficiency in the turbulent transport of heat and water between the land surface and the overlying air Alfieri et al. (2018); Raupach (1994) (you can revisity the [102-Turbulence_and_sensible_heat_flux](#) notebook). Specifically for pyTSEB, vegetation structure and density are also important for estimating wind and radiation extinction through the canopy layer affecting the radiation partitioning Parry et al. (2019) and turbulent transport of momentum, heat and water vapour in the canopy air space Nieto et al. (2018).

In pyTSEB, depending on the plant functional trait, roughness can be estimated using different methods. For short and herbaceous crops, the typical ratio of canopy height (h_c) is applied:

$$d_0 = 2/3h_c z_{0m} = 1/8h_c \quad (4.4)$$

where d_0 is the zero-plane displacement height and z_{0m} is the roughness length for momentum.

For tall and woody canopies, in which leaf density and coverage can play a more important role, pyTSEB can adopt the formulation of Schaudt (2000), which at the same time it relies on the model by Raupach (1994)

See Also

You can check the code implementation in pyTSEB [here](#)

4.3.3.1 Estimate the structural variables based on Earth Observation LAI timeseries

A Land cover map can be used to assign ET model parameters, which are difficult to estimate directly from other Earth Observation data Guzinski et al. (2021). Those parameters, and values assigned to different land cover classes, are listed in Table \ref{tab:lc-params}. Nevertheless, the TSEB-PT model requires all of the parameters, apart from stomatal resistance, while ETLook requires only vegetation height and stomatal resistance . Values of other parameters were adapted from \citet{guzinski_modelling_2020}. In such framework, and for annual plant functional types, canopy height is computed considering its growth as Eq. (5):

$$h_c = h_{min} + (h_{max} - h_{min}) \min\left(\frac{LAI}{LAI_{max}}, 1\right) \quad (4.5)$$

However, in this case, in order to better adjust the obstacle (canopy) height in forests between the different climatic conditions, we propose to use the University of Maryland's Global Forest Canopy Height, 2019. A global 30~m spatial resolution global forest canopy height map developed by combining the Global Ecosystem Dynamics Investigation (GEDI) LiDAR and Landsat time-series Potapov et al. (2021).

IGBP	h_{min}	h_{max}	LAI_{max}	f_c	w_c/h_c	l_w
ENF	GEDI	GEDI	N/A	1	0.5	0.01
DNF	GEDI	GEDI	N/A	1	0.5	0.01
EBF	GEDI	GEDI	N/A	1	1	0.05
DBF	GEDI	GEDI	N/A	1	1	0.1

MF	GED	GED	N/A	1	1	0.1
OSH	1	1	N/A	0.2	1	0.05
CSH	2	2	N/A	1	1	0.05
SAV	8	8	N/A	0.2	1	0.05
WSA	8	8	N/A	0.2	1	0.05
GRA	0	0.25	4	1	1	0.01
CRO	0	2	4	1	1	0.05
WET	1	1	N/A	0.5	1	0.05
CVM	0	8	4	1	1	0.05
BSV	0	0.5	4	0.2	1	0.01
WAT	0	0	N/A	0	0	0

Table 4.2: Land Cover Look-Up-Table for ancillary and structural parameters required by TSEB model, adapted from Guzinski et al. (2025), h_{min} (m) is the minimum canopy height; h_{max} (m) is the maximum canopy height occurring when leaf area index (LAI) reaches its optimal maximum LAI_{max} ; f_c is the at-Nadir fraction of the ground occupied by a clumped canopy ($f_c = 1$ for a homogeneous canopy); w_c/h_c is the canopy shape parameter, representing the canopy width to canopy height ratio; l_w (m) is the average leaf size.

4.3.4 run TSEB-PT

Now we are almost ready to run TSEB-PT. As a summary the workflow below shows all the inputs and outputs generated by TSEB and its submodules:

In addition, we also need to specify the height of measurements of both the wind speed (z_u) and air temperature (z_t), since these two parameters are key to correctly estimate the aerodynamic roughness. These values can be obtained from the site characteristics in [\#site-description](#)

Warning

If you recall from the Eq. 3 of the [102-Turbulence_and_sensible_heat_flux](#) notebook, z_u and z_t are within a logarithmic function and thus these values should **never be lower than the canopy height**, with the risk otherwise to generate arithmetic errors and produce no valid data. In case of tall canopies combined with data from agrometeorological stations (usually with sensors at 2m) it is recommended to drive these values to be well above the canopy, ideally above the roughness sublayer (e.g. $2 \times h_c$).

As an example on how TSEB finds iteratively one plausible solution (i.e. non-negative latent heat fluxes during daytime) we will run one single case and plot the different values during the iterative process:

4.3.4.1 Satellite products

These values are just a sample extracted from ECOSTRESS LST product and biophysical traits derived from Sentinel-2.

```
lst = 314.58 # ECOSTRESS Land Surface Temperature (K)
vza = 9.4321671 # ECOSTRESS View Zenith Angle (deg.)
sza = 28.6739051703528 # ECOSTRESS Sun Zenith Angle (deg.)
lai = 0.89952921293024 # Sentinel-2 Leaf Area Index
leaf_angle = 57.5344928614439 # Sentinel-2 Mean Leaf Inclination Angle (deg.)
f_g = 0.826559189567835 # Sentinel-2 Fraction of LAI that is green
print("Parsed Earth Observation inputs correctly, you can continue to next cell")
```

Parsed Earth Observation inputs correctly, you can continue to next cell

4.3.4.2 Weather forcing

Weather forcing is extracted from the [ECMWF ERA5 hourly reanalysis](#), which are interpolated and topographically corrected to get the weather forcings at the time of the satellite overpass.

See Also

The weather forcing preprocessing is done using the [meteo_utils GitHub package](#)

```
t_air = 302.309448242188 # air temperature (K)
ea = 14.79697608947758 # Vapour pressure (hPa)
ws = 4.32157111167908 # Wind speed (ms-1)
press = 980.325317382812 # Atmospheric pressure (hPa)
lw_in = 332.861083984375 # Longwave irradiance (Wm-2)
sw_in = 1014.41126346588 # Shortwave irradiance (Wm-2)

sn_c = 500.114215358855 # Estimated canopy net shortwave radiation (Wm-2)
sn_s = 259.319271056837 # Estimated soil net shortwave radiation (Wm-2)

Z_T = 100 # Temperature measurement at blending height
Z_U = 100 # Wind measurement at blending height

print("Parsed weather forcing correctly, you can continue to next cell")
Parsed weather forcing correctly, you can continue to next cell
```

4.3.4.3 Structural canopy parameters

```
f_c = 1 # Homogeneous canopy
w_c = 1 # Homogeneous canopy
leaf_width = 0.01 # Effective leaf size (m)
h_c = 0.0791470505387601 # Canopy height (m)
z0_soil = 0.001 # Soil/background aerodynamic roughness

print("Parsed ancillary canopy structural parameters correctly, you can
continue to next cell")
Parsed ancillary canopy structural parameters correctly, you can continue to
next cell
```

4.3.5 Initial state variables and constants

```
resistance_method = [0]
resistance_params = {}
# Standard PT value
alpha_pt_0 = 1.26

# Thermal spectra
e_v = 0.99 # Leaf emissivity
e_s = 0.94 # Soil emissivity

# Calculate constants
rho = TSEB.met.calc_rho(press, ea, t_air) # Air density
c_p = TSEB.met.calc_c_p(press, ea) # Heat capacity of air
[z_0m, d_0] = TSEB.res.calc_roughness(lai,
                                     h_c,
                                     w_c,
                                     TSEB.res.CROP,
                                     f_c=np.array(f_c))

x_lad = TSEB.rad.leafangle_2_chi(leaf_angle)
z_0h = TSEB.res.calc_z_0h(z_0m, kB=0) # Roughness length for heat transport

# Calculate LAI dependent parameters for dataset where LAI > 0
omega0 = TSEB.CI.calc_omega0_Kustas(lai, f_c, x_LAD=x_lad, isLAIeff=True)
F = np.asarray(lai / f_c) # Real LAI
```

```
# Fraction of vegetation observed by the sensor
f_theta = TSEB.calc_F_theta_campbell(vza, F, w_C=w_c, Omega0=omega0,
x_LAD=x_lad)

print("Computed initial TSEB state variables, you can continue to next cell")

Computed initial TSEB state variables, you can continue to next cell
```

4.3.6 Monin-Obukhov iteration and Priestley Taylor iteration

```
# Create the arrays to store the different values during the iterations
t_c_array = []
t_s_array = []
le_c_array = []
le_s_array = []
h_c_array = []
h_s_array = []
t_aero_array = []
alpha_pt_array = []
l_mo_array = []
l_diff_array = []

L_MO_THRES = 0.001
```

```
def calc_L_n_Campbell(T_C, T_S, L_dn, lai, emisVeg, emisGrd, x_LAD=1):
    ''' Net longwave radiation for soil and canopy layers
```

```
    Estimates the net longwave radiation for soil and canopy layers using
    based on equation 2a
    from [Kustas1999]_ and incorporated the effect of the Leaf Angle
    Distribution based on [Campbell1998]_
```

Parameters

T_C : float

Canopy temperature (K).

T_S : float

Soil temperature (K).

L_dn : float

Downwelling atmospheric longwave radiation (W m⁻²).

lai : float

Effective Leaf (Plant) Area Index.

emisVeg : float

Broadband emissivity of vegetation cover.

emisGrd : float

Broadband emissivity of soil.

x_LAD: float, optional

x parameter for the ellipsoidal Leaf Angle Distribution function,
use x_LAD=1 for a spherical LAD.

Returns

L_nC : float

Net longwave radiation of canopy (W m⁻²).

L_nS : float

Net longwave radiation of soil (W m⁻²).

References

```
.. [Kustas1999] Kustas and Norman (1999) Evaluation of soil and vegetation
heat
    flux predictions using a simple two-source model with radiometric
temperatures for
    partial canopy cover, Agricultural and Forest Meteorology, Volume 94,
Issue 1,
    Pages 13-29, http://dx.doi.org/10.1016/S0168-1923\(99\)00005-2.
```

'''

```
def calc_stephan_boltzmann(t_k):
    '''Calculates the total energy radiated by a blackbody.

    Parameters
    -----
    t_k : float
        body temperature (Kelvin)

    Returns
    -----
    M : float
        Emitted radiance (W m-2)'''
    # Stephan Boltzmann constant (W m-2 K-4)
    sb = 5.670373e-8
    m = sb * t_k**4
    return m

from pyTSEB.net_radiation import calc_spectra_Campbell
# calculate long wave emissions from canopy, soil and sky
L_C = emisVeg * calc_stephan_boltzmann(T_C)
L_S = emisGrd * calc_stephan_boltzmann(T_S)
# Calculate the canopy spectral properties
_, albl, _, taudl = calc_spectra_Campbell(lai,
                                          0,
                                          1.0 - emisVeg,
                                          0,
                                          1.0 - emisGrd,
                                          x_lad=x_LAD,
                                          lai_eff=None)

# calculate net longwave radiation divergence of the soil
L_nS = emisGrd * taudl * L_dn + emisGrd * (1.0 - taudl) * L_C - L_S
L_nC = (1 - albl) * (1.0 - taudl) * (L_dn + L_S) - 2.0 * (1.0 - taudl) *
L_C
return L_nC, L_nS

def calc_t_s(lst, t_c, f_theta):
    '''Estimates soil temperature from the directional LST.

    Parameters
    -----
    lst : float
        Directional Radiometric Temperature (K).
    t_c : float
        Canopy Temperature (K).
    f_theta : float
        Fraction of vegetation observed.

    Returns
    -----
    t_s: float
        Soil temperature (K).

    References
    -----
    Eq. 1 in [Norman1995]_'''

    t_temp = lst**4 - f_theta * t_c**4
    t_s = (t_temp / (1.0 - f_theta))**0.25
    return t_s

# Assume neutral conditions
l_mo = np.array(np.inf)
```

```

l_old = float(l_mo)
u_friction = TSEB.M0.calc_u_star(ws, Z_U, l_mo, d_0, z_0m)
# First assume that canopy temperature equals the minimum of Air or
# radiometric T
t_c = np.minimum(lst, t_air)
# Retrieve soil temperature from LST and Tc based on gap fraction
t_s = calc_t_s(lst, t_c, f_theta)
# First guess of aerodynamic temperature is t air
t_ac = float(lst)

l_diff = np.inf
for n_iterations in range(50):
    alpha_pt_rec = alpha_pt_0 + 0.1
    le_s = -1
    while le_s < 0:
        # Reduce the Priestley Taylor parameter by 0.1
        alpha_pt_rec -= 0.1

    # Calculate aerodynamic resistances
    r_a, r_x, r_s = TSEB.calc_resistances(
        resistance_method,
        {"R_A": {"z_T": Z_T, "u_friction": u_friction, "L": l_mo,
                  "d_0": d_0, "z_0H": z_0h},
         "R_x": {"u_friction": u_friction, "h_C": h_c,
                  "d_0": d_0,
                  "z_0M": z_0m, "L": l_mo, "F": F, "LAI": lai,
                  "leaf_width": leaf_width,
                  "z0_soil": z0_soil,
                  "massman_profile": [0, []],
                  "res_params": {k: resistance_params[k] for k in
resistance_params.keys()}},
         "R_S": {"u_friction": u_friction, "h_C": h_c,
                  "d_0": d_0,
                  "z_0M": z_0m, "L": l_mo, "F": F, "omega0": omega0,
                  "LAI": lai, "leaf_width": leaf_width,
                  "z0_soil": z0_soil, "z_u": Z_U,
                  "deltaT": t_s - t_ac, 'u': ws, 'rho': rho,
                  "c_p": c_p, "f_cover": f_c, "w_C": w_c,
                  "massman_profile": [0, []],
                  "res_params": {k: resistance_params[k] for k in
resistance_params.keys()}},
        )

    # Calculate net longwave radiation with current values of T_C and T_S
    ln_c, ln_s = calc_L_n_Campbell(
        t_c, t_s, lw_in, lai, e_v, e_s, x_LAD=x_lad)
    rn_c = sn_c + ln_c
    rn_s = sn_s + ln_s
    # Compute Soil Heat Flux Ratio
    g_flux = 0.35 * rn_s

    # Calculate the canopy and soil temperatures using the Priestley
    # Taylor approach
    h_c_flux = TSEB.calc_H_C_PT(
        rn_c, f_g, t_air, press, c_p, alpha_pt_rec)

    t_c = TSEB.calc_T_C_series(
        lst, t_air, r_a, r_x, r_s, f_theta, h_c_flux, rho, c_p)

    # Calculate soil temperature
    t_s = calc_t_s(lst, t_c, f_theta)

    # Recalculate soil aerodynamic resistance
    _, _, r_s = TSEB.calc_resistances(
        resistance_method,

```



```

{
    "R_S": {"u_friction": u_friction, "h_C": h_c,
            "d_0": d_0,
            "z_0m": z_0m, "L": l_mo, "F": F, "omega0": omega0,
            "LAI": lai, "leaf_width": leaf_width,
            "z0_soil": z0_soil, "z_u": Z_U,
            "deltaT": t_s - t_ac, 'u': ws, 'rho': rho,
            "c_p": c_p, "f_cover": f_c, "w_C": w_c,
            "massman_profile": [0, []],
            "res_params": {k: resistance_params[k] for k in
resistance_params.keys()}}
    }
)

# Get air temperature at canopy interface
t_ac = ((t_air / r_a + t_s / r_s + t_c / r_x)
        / (1.0 / r_a + 1.0 / r_s + 1.0 / r_x))

# Calculate soil fluxes
h_s_flux = rho * c_p * (t_s - t_ac) / r_s

# Estimate latent heat fluxes as residual of energy balance at the
# soil and the canopy
le_s = rn_s - g_flux - h_s_flux
le_c = rn_c - h_c_flux

# No tranpiration
if alpha_pt_rec <= 0.0:
    h_s_flux = np.minimum(h_s_flux, rn_s - g_flux)
    g = np.maximum(g_flux, rn_s - h_s_flux)
    le_s = 0
    le_c = 0

# Calculate total fluxes
h = h_c_flux + h_s_flux
le = le_c + le_s

t_c_array.append(t_c)
t_s_array.append(t_s)
le_c_array.append(le_c)
le_s_array.append(le_s)
h_c_array.append(h_c_flux)
h_s_array.append(h_s_flux)
t_aero_array.append(t_ac)
alpha_pt_array.append(alpha_pt_rec)
l_mo_array.append(z_0m / l_mo)

# Recompute the M0 length
l_mo = TSEB.M0.calc_L(
    u_friction, t_air, rho, c_p, h, le)
u_friction = TSEB.M0.calc_u_star(
    ws, Z_U, l_mo, d_0, z_0m)

l_diff = np.fabs(l_mo - l_old) / np.fabs(l_old)
l_diff_array.append(l_diff)

l_diff = np.fabs(l_mo - l_old) / np.fabs(l_old)
l_old = float(l_mo)
if l_diff <= L_MO_THRES:
    break

iterations = np.arange(len(le_c_array))

[t_c_array, t_s_array, le_c_array, le_s_array, h_c_array,

```

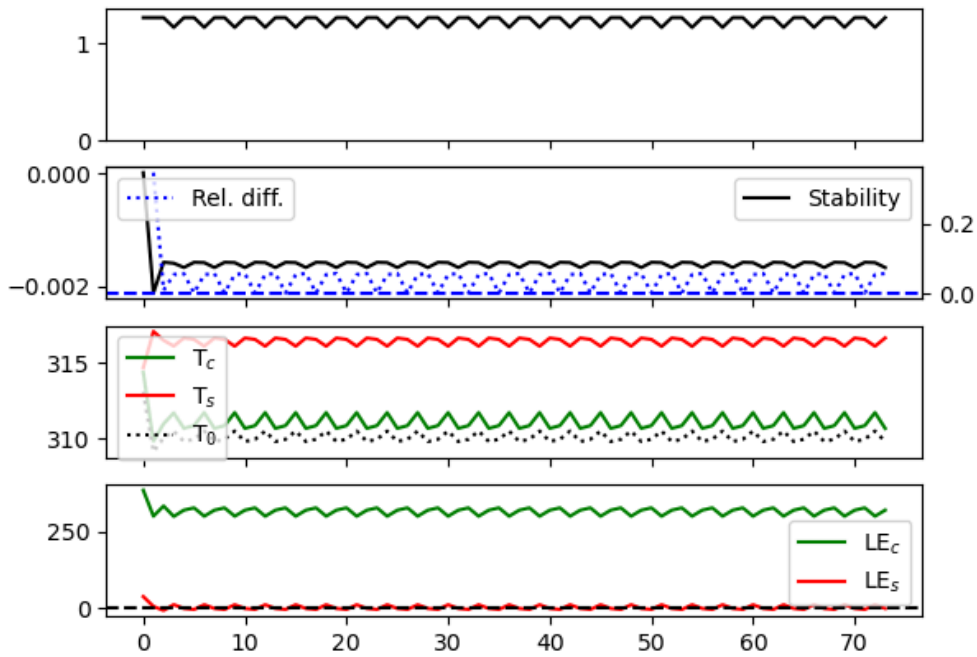
```
h_s_array, t_aero_array, alpha_pt_array, l_mo_array, l_diff_array] = map(
    np.array, [t_c_array, t_s_array, le_c_array, le_s_array, h_c_array,
    h_s_array, t_aero_array, alpha_pt_array, l_mo_array, l_diff_array])
```

```
fig, axs = plt.subplots(nrows=4, ncols=1, sharex=True)
```

```
axs[0].plot(iterations, alpha_pt_array, "k-", label=r"$\alpha_{PT}$")
axs[0].set_ylim([0, 1.36])
```

```
axs[1].plot(iterations, l_mo_array, "k-", label=r"Stability")
ax = axs[1].twinx()
ax.plot(iterations, l_diff_array, "b:", label="Rel. diff.")
ax.axhline(y=L_MO_THRES, color="b", ls="--")
axs[1].legend(loc="upper right")
ax.legend(loc="upper left")
axs[2].plot(iterations, t_c_array, "g-", label=r"$T_c$")
axs[2].plot(iterations, t_s_array, "r-", label=r"$T_s$")
axs[2].plot(iterations, t_aero_array, "k:", label=r"$T_{\theta}$")
axs[2].legend()
axs[3].plot(iterations, le_c_array, "g-", label=r"$LE_c$")
axs[3].plot(iterations, le_s_array, "r-", label=r"$LE_s$")
axs[3].axhline(y=0, ls="--", color="k")
axs[3].legend()
```

```
<matplotlib.legend.Legend at 0x71cbe241cdd0>
```



4.3.7 Extrapolation to daily values

TSEB-PT directly estimates instantaneous turbulent fluxes at the LST acquisition time (in $W m^{-2}$). However most applications require daily estimates of evapotranspiration (e.g. in $mm day^{-1}$).

One simple approach to extrapolate the instantaneous latent heat fluxes ($\lambda E_{inst.}$) to daily ET rates is by assuming that the ratio of instantaneous to daily shortwave irradiance ($\frac{S_{day}^{down}}{S_{inst.}^{down}}$) is preserved during daytime hours. Based on this assumption daily latent heat flux can be estimated using (6):

$$\lambda E_{day} = \lambda E_{inst.} \frac{S_{day}^{down}}{S_{inst.}^{down}} \quad (4.6)$$

And then the daily latent heat flux (W m^{-2}) is converted water units (mm day^{-1}) by the latent heat of vaporization.

```
ta_day = 298.563903808594 # Average daily temperature (K)
sw_in_day = 335.073944091797 # Average shortwave irradiance ( $\text{Wm}^{-2}$ )
```

```
le_daily = le * sw_in_day / sw_in
et = TSEB.met.flux_2_evaporation(le_daily, ta_day, 24)
```

```
print(f"Daily ET is estimatet at {et:.2f} mm day-1")
```

Daily ET is estimatet at 3.74 mm day⁻¹

4.4 Satellite-based implementation of TSEB

We will now show an example on how to run pyTSEB with satellite imagery. We will run the model with an example imagery from the Sentinel constellation around (2km x 2km) the [Majadas](#) EC towers (ES-LMa/ES-LM1/ES-LM2).

For this example, we will only use the land surface temperature (LST) and leaf area index (LAI) products from the Sentinel constellation. The LST was obtained from Sentinel-3 SLSTR sensor and was sharpened to 20m using the Sentinel-2 MSI sensor and a Data Mining Sharpening algorithm ([pyDMS](#)). While the LAI products were estimated by inverting a radiative transfer model (i.e. [pyProSail](#)) using a hybrid appraoch. These processing algorithms are described in [Guzinski et al. 2023](#) and [Guzinski et al. \(2025\)](#).

Normally, other products such land cover, green fraction, canopy height (i.e. GEDI) maps and other spatially explicit variables can also be intgrated into the workflow but for this example we keep it simple.

See Also

For more in depth tutorials on how pre-process cloud-based Sentinel inputs and run TSEB, please see [SenET toolbox](#)

4.4.1 Open satellite inputs

We have prepared inputs of LST and LAI images from 2022-04-16 over ES-LMa (2x2km around tower)

```
from osgeo import gdal
input_dir = Path() / "input"
sat_dir = input_dir / 'satellite'
# open lst
lst_file = sat_dir / '20220416T103841_S3-DMS_LST.tif'
lst_fid = gdal.Open(str(lst_file))
lst_ar = lst_fid.GetRasterBand(1).ReadAsArray()
# open lai
lai_file = sat_dir / '20220416_S2_LAI.tif'
lai_fid = gdal.Open(str(lai_file))
lai_ar = lai_fid.GetRasterBand(1).ReadAsArray()

s3_overpass_datetime = pd.to_datetime('2022-04-16 10:30') # time in UTC

fig = make_subplots(
    rows=1,
    cols=2,
    horizontal_spacing=0.03,
    subplot_titles=("LST", "LAI"))

# LST
fig.add_trace(
    go.Heatmap(
        z=lst_ar,
        colorscale="RdYlBu_r",
```

```

    zmin=295,
    zmax=305,
    colorbar=dict(
        title="LST (K)",
        orientation="h",
        x=0.2,
        y=-0.25,
        len=0.5,
        thickness=15)),
    row=1,col=1)

# LAI
fig.add_trace(
    go.Heatmap(
        z=lai_ar,
        zmin=0,
        zmax=4,
        colorscale="Viridis",
        colorbar=dict(
            title="LAI (m²/m²)",
            orientation="h",
            x=0.72,
            y=-0.25,
            len=0.5,
            thickness=15)),
    row=1, col=2)

fig.update_layout(
    title=f"Sentinel inputs from {s3_overpass_datetime}",
    height=500,
    width=1000)

# Make pixels square like imshow
fig.update_yaxes(autorange="reversed", scaleanchor="x", row=1, col=1)
fig.update_yaxes(autorange="reversed", scaleanchor="x2", row=1, col=2)

```

4.4.2 Meteo and ancillary data from EC tower

Since we are use the example from Majadas, we will acquire meteorological and ancillary data directly form tower data (i.e. ES-LMa)

4.4.2.1 Get Majadas site data

```

site = 'ES-LMa'
# Measurement height of wind speed and air temperature (in the case of ES-LMa,
it is 15m)
z_u = 15.
z_t = 15.
# Site metadata file
site_metadata_file = input_dir / "sites.csv"
site_metadata = pd.read_csv(site_metadata_file, sep=";")
site_id = site_metadata["SITE_ID"] == site
lat = site_metadata.loc[site_id, "LOCATION_LAT"].item()
lon = site_metadata.loc[site_id, "LOCATION_LONG"].item()
elev = site_metadata.loc[site_id, "LOCATION_ELEV"].item()
utc_offset = site_metadata.loc[site_id, "UTC_OFFSET"].item()
stdlon = 15 * utc_offset
site_metadata.loc[site_id]

```

	SITE_ID	IGBP	LOCATION_LAT	LOCATION_LONG	LOCATION_ELEV	UTC_OFFSET
113	ES-LMa	SAV	39.94033	-5.77465	265.0	1.0

4.4.2.2 Get Majadas weather forcing

```

ec_dir = input_dir / "eddy_covariance"
# Set the input files to Majadas tower (ES-LMa)

```

```

ec_filename = ec_dir / f"FLX_{site}_FLUXNET_SUBSET_HH.csv"

# Read the fluxnet EC table and preprocess
ec_maj = pd.read_csv(ec_filename, sep=',', na_values=-9999)
ec_maj["TIMESTAMP_START"] = pd.to_datetime(
    ec_maj["TIMESTAMP_START"], format="%Y%m%d%H%M")

# satellite overpass mask
datetime_mask = ec_maj['TIMESTAMP_START'] == s3_overpass_datetime +
pd.Timedelta(hours=utc_offset)
# extract meteo data at overpass time
Ta = float(ec_maj['TA_F'].values[datetime_mask][0]) + 273.15 # air temperature
(convert to K)
sw_in = float(ec_maj['SW_IN_F'].values[datetime_mask][0]) # incoming
shortwave irradiance
lw_in = float(ec_maj['LW_IN_F'].values[datetime_mask][0]) # incoming longwave
irradiance
vpd = float(ec_maj['VPD_F'].values[datetime_mask][0]) # vapour pressure
deficit
es = TSEB.met.calc_vapor_pressure(Ta) # saturated vapour pressure
ea = es - vpd # actual vapour pressure
u = float(ec_maj['WS_F'].values[datetime_mask][0]) # wind speed
P = float(ec_maj['PA_F'].values[datetime_mask][0]) * 10 # air pressure
convert to hPa

```

```

print(f'Meteo conditions at {site} during S3 overpass
({s3_overpass_datetime}):\\n'
      f"TA = {Ta:.1f} K\\n"
      f"SW_IN = {sw_in:.0f} W m-2\\n"
      f"EA = {ea:.1f} hPa\\n"
      f"WS = {u:.2f} ms-1")

```

Meteo conditions at ES-LMa during S3 overpass (2022-04-16 10:30:00):

TA = 296.4 K
 SW_IN = 839 W m⁻²
 EA = 14.6 hPa
 WS = 0.87 ms⁻¹

4.4.3 Biophysical and structural parameters

For simplicity, we will assume a constant f_c and h_c across the image window. In other applications, other spatial or satellite-based estimates could also be used to inform these variables such as tree fractional cover products (e.g., canopy height models from LiDAR such as GEDI, see Table 1).

```

# from site information
fc_ar = np.full_like(lst_ar, 1) # fractional cover
hc_ar = np.full_like(lst_ar, 8) # canopy height

# sun angles during S3 overpass
sza, saa = TSEB.met.calc_sun_angles(lat,
                                     lon,
                                     stdlon,
                                     s3_overpass_datetime.dayofyear,
                                     s3_overpass_datetime.hour +
s3_overpass_datetime.minute / 60.)
# local/real LAI of overstory
F = lai_ar / fc_ar
# clumping of overstory
omega_0 = TSEB.CI.calc_omega0_Kustas(lai_ar, fc_ar, x_LAD=1)
omega = TSEB.CI.calc_omega_Kustas(omega_0, np.minimum(sza, 90), w_C=w_c)
# effective LAI of overstory
lai_eff = F * omega

## roughness parameters

```

```
[z_0m, d_0] = TSEB.res.calc_roughness(lai_ar,
                                     hc_ar,
                                     np.full_like(lai_ar, w_c),
                                     np.full_like(lai_ar,
TSEB.res.BROADLEAVED_E),
                                     f_c=fc_ar)

# Ensure realistic values
d_0[d_0 < 0] = 0
z_0m[np.isnan(z_0m)] = z0_soil
d_0[d_0 < z0_soil] = z0_soil

print("Parsed biophysical and structural parameters correctly, you can
continue to next cell")

Parsed biophysical and structural parameters correctly, you can continue to
next cell
```

4.4.4 Running pyTSEB with satellite inputs

```
vza = np.full_like(lst_ar, 21.88) # from S3-SLSTR product

print('Estimating net shortwave radiation [...]\\n')

# Estimates the direct and diffuse solar radiation in NIR and VIS
difvis, difnir, fvis, fnir = TSEB.rad.calc_diffuse_ratio(np.full_like(lst_ar,
sw_in),
                                                         np.full_like(lst_ar,
sza),
press=np.full_like(lst_ar, P))

par_dir = fvis * (1. - difvis) * np.full_like(lst_ar, sw_in)
nir_dir = fnir * (1. - difnir) * np.full_like(lst_ar, sw_in)
par_dif = fvis * difvis * np.full_like(lst_ar, sw_in)
nir_dif = fnir * difnir * np.full_like(lst_ar, sw_in)

# shortwave radiation model
sn_c, sn_s = TSEB.rad.calc_Sn_Campbell(lai_ar,
                                     np.full_like(lst_ar, sza),
                                     par_dir + nir_dir,
                                     par_dif + nir_dif,
                                     fvis,
                                     fnir,
                                     np.full_like(lst_ar, 0.05),
                                     np.full_like(lst_ar, 0.08),
                                     np.full_like(lst_ar, 0.26),
                                     np.full_like(lst_ar, 0.33),
                                     np.full_like(lst_ar, 0.07),
                                     np.full_like(lst_ar, 0.07),
                                     x_LAD=1,
                                     LAI_eff=lai_eff)

# use Norman and Kustas 1995 resistance framework
Resistance_flag=[0,{}]
# using constant ratio approach to estimate G
G_constant = 0.35
calcG = [[1], G_constant]

print('Running TSEB [...]\\n')

[flag_PT_all, T_soil, T_veg, T_AC, Ln_S, Ln_C, LE_C, H_C,
LE_S, H_S, G_mod, R_S, R_X, R_A, u_friction, L, n_iterations] =
TSEB.TSEB_PT(lst_ar,
vza,
np.full_like(lst_ar, Ta),
np.full_like(lst_ar, u),
```

```

np.full_like(lst_ar, ea),
np.full_like(lst_ar, P),
sn_c,
sn_s,
np.full_like(lst_ar, lw_in),
lai_ar,
hc_ar,
e_v,
e_s,
z_0m,
d_0,
z_t,
z_u,
leaf_width=leaf_width,
alpha_PT=alpha_pt_0,
f_c=fc_ar,
f_g=np.full_like(lst_ar,0.8),
calcG_params=calcG,
resistance_form=Resistance_flag)

# save outputs in outdict
LE = LE_C + LE_S
H = H_C + H_S

Rn_C = Ln_C + sn_c
Rn_S = Ln_S + sn_s
Rn = Rn_C + Rn_S

fig = make_subplots(
    rows=1,
    cols=3,
    horizontal_spacing=0.03,
    subplot_titles=("LE", "H", "Rn"))

# LE
fig.add_trace(
    go.Heatmap(
        z=LE,
        colorscale="Blues",
        zmin=100,
        zmax=500,
        colorbar=dict(
            title="LE (W/m²)",
            orientation="h",
            x=0.13,
            y=-0.2,
            len=0.3,
            thickness=15)),
    row=1,col=1)

# H
fig.add_trace(
    go.Heatmap(
        z=H,
        zmin=100,

```

```

zmax=500,
colorscale="Reds",
colorbar=dict(
    title="H (W/m²)",
    orientation="h",
    x=0.48,
    y=-0.2,
    len=0.3,
    thickness=15)),
row=1, col=2)

# Rn
fig.add_trace(
    go.Heatmap(
        z=Rn,
        zmin=200,
        zmax=800,
        colorscale="Oranges",
        colorbar=dict(
            title="Rn (W/m²)",
            orientation="h",
            x=0.82,
            y=-0.2,
            len=0.3, # colorbar length
            thickness=15)),
        row=1, col=3)

fig.update_layout(
    title="Modelled instantaneous fluxes from TSEB",
    height=500,
    width=1500)

# Make pixels square like imshow
fig.update_yaxes(autorange="reversed", scaleanchor="x", row=1, col=1)
fig.update_yaxes(autorange="reversed", scaleanchor="x2", row=1, col=2)
fig.update_yaxes(autorange="reversed", scaleanchor="x3", row=1, col=3)

Estimating net shortwave radiation [...]

Running TSEB [...]

Iteration: 0, non-converged pixels: 9914, max L diff: inf, total time:
0.000124, loop time: 0.000124
Iteration: 1, non-converged pixels: 9914, max L diff: inf, total time:
0.027657, loop time: 0.027533
Iteration: 2, non-converged pixels: 9914, max L diff: 3.496068, total time:
0.056905, loop time: 0.029248
Iteration: 3, non-converged pixels: 9914, max L diff: 0.854417, total time:
0.086136, loop time: 0.029231
Iteration: 4, non-converged pixels: 9749, max L diff: 0.373019, total time:
0.116538, loop time: 0.030402

Iteration: 5, non-converged pixels: 9304, max L diff: 0.185218, total time:
0.144042, loop time: 0.027504
Iteration: 6, non-converged pixels: 9281, max L diff: 0.241179, total time:
0.166857, loop time: 0.022815
Iteration: 7, non-converged pixels: 9124, max L diff: 0.203209, total time:
0.189492, loop time: 0.022634

Iteration: 8, non-converged pixels: 9096, max L diff: 0.271055, total time:
0.211783, loop time: 0.022292
Iteration: 9, non-converged pixels: 6895, max L diff: 0.222894, total time:
0.234346, loop time: 0.022563
Iteration: 10, non-converged pixels: 1216, max L diff: 0.305234, total time:
0.245586, loop time: 0.011239
Iteration: 11, non-converged pixels: 738, max L diff: 0.244274, total time:

```



```

0.250536, loop time: 0.004951
Iteration: 12, non-converged pixels: 236, max L diff: 0.344506, total time:
0.254630, loop time: 0.004093
Iteration: 13, non-converged pixels: 64, max L diff: 0.267469, total time:
0.257930, loop time: 0.003301
Iteration: 14, non-converged pixels: 7, max L diff: 0.389849, total time:
0.261097, loop time: 0.003167
  
```

Questions:

- How do the modelled fluxes compare to the tower measurements during overpass time?
- How could we get a map of daily ET (mm/day) from these instantaneous fluxes?

4.5 Three-Source Energy Balance (3SEB) model adaptation

The Three-Source Energy Balance (3SEB) model (BurchardLevine et al. (2021)) is an adapted version of the TSEB model Norman et al. (1995), where an additional vegetation source is incorporated into TSEB to better characterize agro-ecosystems with multiple vegetation layers that have distinct structural and physiological features. This adaptation was initially envisioned for savanna-like ecosystems, which are characterized by the co-existence of tree and herbaceous species with very different structural and phenological dynamics.

However, the 3SEB model structure is also well adapted for perennial tree crops that have cover crops in the interrow, such as vineyards, olive orchards, almonds etc. In principle, the 3SEB model is more suited to represent the complex interactions between a clumped vegetation source planted in rows with an additional herbaceous vegetation source located in the interrows.

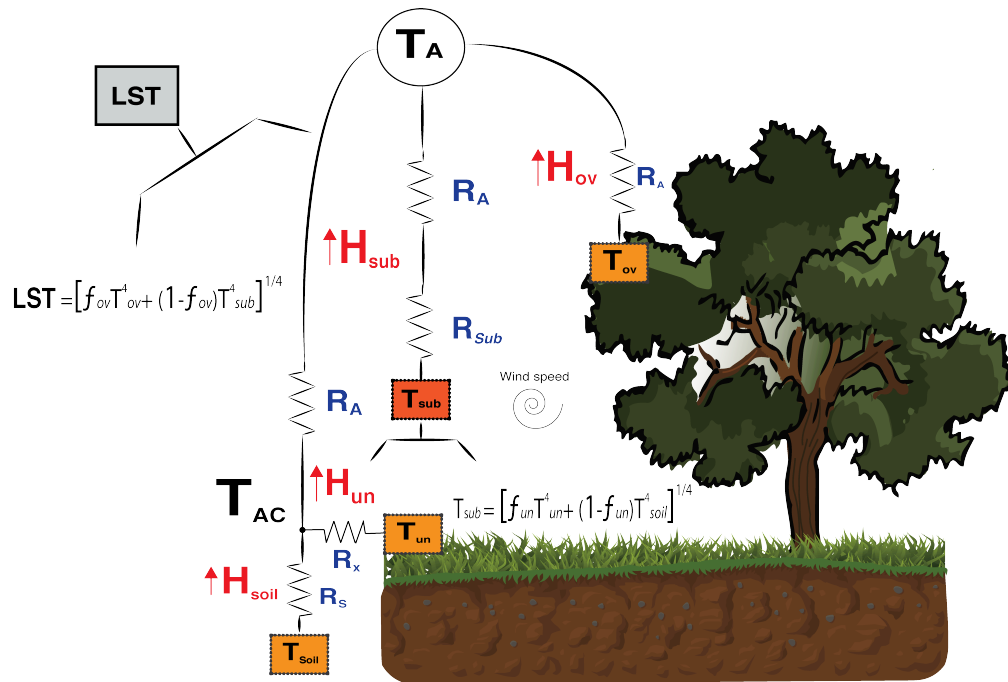


Figure 4.3: Three-Source energy balance scheme for transport sensible heat (H). From BurchardLevine et al. (2021)

In 3SEB, the energy balance is therefore separated between the vegetation overstory (ov), vegetation understory (un) and soil (s) sources as:

$$\begin{aligned}
 R_n &\approx H + \lambda E + G \\
 R_{n,ov} &\approx H_{ov} + \lambda E_{ov} \\
 R_{n,un} &\approx H_{un} + \lambda E_{un} \\
 R_{n,s} &\approx H_s + \lambda E_s + G
 \end{aligned}
 \tag{4.7}$$

The energy balance equations are simplified as they neglect additional components of the energy balance that are usually neglected, such as heat advection, storage of energy in the canopy layer or energy for the fixation of CO₂, which are not computed by the model.

4.5.1 3SEB inputs

3SEB has very similar data requirements to TSEB (as discussed above) but a key distinction is the need to separate biophysical properties (e.g., LAI and H_c) for the two vegetation sources instead of only one vegetation source as in TSEB.

4.5.2 Structural properties for each vegetation layer

Here we will characterize each vegetation (e.g. trees and grasses) with their respective values

```
# fractional cover
f_c_ov = 0.2 # overstory fractional cover (e.g., clumped trees)
f_c_un = 1. # understory fractional cover (e.g., homogeneous grass)

# leaf width
leaf_width_ov = 0.05 # Effective leaf size (m) of overstory (e.g. trees)
leaf_width_un = 0.01 # Effective leaf size (m) of understory (e.g. grass)
w_c_ov = 1
w_c_un = 1
# canopy height
h_c_ov = 8.0 # Canopy height (m) of overstory (e.g. trees)
h_c_un = 0.25 # Canopy height (m) of understory (e.g. grass)
print("Parsed ancillary canopy structural parameters correctly, you can
continue to next cell")
```

Parsed ancillary canopy structural parameters correctly, you can continue to next cell

4.5.2.1 LAI and f_g decomposition between vegetation layers

In a savanna ecosystem, we would need to characterize the LAI for both the trees (i.e. overstory) and grasses (i.e. understory) or, in vineyards for example, the grapevine (i.e. overstory) and the cover crop (i.e. understory). There is no general formula to separate or unmix LAI from the different vegetation sources, there are potentially many different methods that can be used to disentangle the biophysical properties of both vegetation layers (e.g. unmixing algorithms, radiative transfer modeling etc)

In this example, we will apply 3SEB over an Iberian savanna-like ecosystem (known as Dehesas in Spain or Montados in Portugal). These ecosystems are characterized by the co-existence of scattered trees (mostly Holm Oaks or *Quercus Ilex L*) over annual herbaceous species. The decomposition of features between both vegetation elements is challenging in savannas, as they overlap over each other and cannot be discriminated at the conventional satellite spatial resolution (>10m) or EC tower footprint. However, the two vegetation layers have distinct phenological patterns and these temporal differences can be used to infer the LAI of each source. During the dry period, the lack of water availability causes the understory species to senesce, while the overstory, largely composed of evergreen trees with deeper rooting systems, are adapted to survive these arid conditions. As such, during the seasonal drought, the landscape spectral signal and green LAI (i.e., LAI_{eco}) is mostly composed of the overstory (LAI_{ov}), capturing little to none of the LAI_{un} . Therefore, for each year, the LAI_{eco} from the peak dry period can be considered as LAI_{ov} and for sites with evergreen tree species, a year-round constant LAI_{ov} can be implemented as the LAI from the tree species do not fluctuate greatly and have minimal phenology (i.e. Luo et al. (2018); Moore et al. (2016).

```
bio_dir = input_dir / "canopy"
bio_filename = bio_dir / f"{site}_HLS-l2c.csv"
print(f"Biophysical traits file path is {bio_filename}")

# Read the fluxnet EC table and preprocess
ec = pd.read_csv(ec_filename, sep=',', na_values=-9999)
```

```

ec["TIMESTAMP_START"] = pd.to_datetime(
    ec["TIMESTAMP_START"], format="%Y%m%d%H%M")

ec["TIMESTAMP_END"] = pd.to_datetime(
    ec["TIMESTAMP_END"], format="%Y%m%d%H%M")

new_cols = {}
new_cols["TIMESTAMP"] = pd.to_datetime(
    ec[["TIMESTAMP_START", "TIMESTAMP_END"]].mean(axis=1))

new_cols["DATE"] = new_cols["TIMESTAMP"].dt.date
ec = pd.concat([ec, pd.DataFrame(new_cols)], axis=1)

# Read the satellite biophysical traits table
bio = pd.read_csv(bio_filename, sep=";", na_values=-9999)
bio["DATE"] = pd.to_datetime(bio["DATE"], format="%Y-%m-%d").dt.date

# Merge both tables by date
ec = ec.merge(bio, on="DATE")
# Ecosystem LAI
lai_eco = ec['LAI'].values

ec.loc[:, "F_C"] = f_c_ov

# initialize overstory LAI_ov
lai_ov = np.zeros_like(lai_eco)

years = pd.to_datetime(ec['DATE'].values).year

# get lai_ov for each year
for year in np.unique(years):
    lai_year = ec['LAI'].values[years == year]
    # lowest 10% percentile as estimate of LAI_ov (during summer period)
    lai_min = np.percentile(lai_year, 10)
    lai_ov[years == year] = lai_min

# overstory LAI
ec['LAI_ov'] = lai_ov
# understory LAI
lai_un = (ec['LAI'].values - ec['LAI_ov'].values)/(1-f_c_ov) # (account for
LAI_un below tree canopy)
lai_un[lai_un<0.3] = 0.3
ec['LAI_un'] = lai_un

print('Plotting LAI decomposition time series [...]')
fig = make_subplots(rows=1, cols=1,
                    shared_xaxes=True,
                    horizontal_spacing=0.01,
                    subplot_titles=("LAI", "Leaf pigments"))

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI"],
                           name="LAI_eco", mode="lines", line={"color":
"black"}),
              row=1, col=1)

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI_ov"],
                           name="LAI_ov", mode="lines", line={"color": "green",
"dash": "dash"}),
              row=1, col=1)

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI_un"],
                           name="LAI_un", mode="lines", line={"color":
"indianred", "dash": "dash"}),
              row=1, col=1)

```

```
fig.update_xaxes(title_text="Date", row=3, col=1)
fig.update_layout(title_text=f"Biophysical traits timeseries in {site}")
Biophysical traits file path is input/canopy/ES-LMa_HLS-l2c.csv
Plotting LAI decomposition time series [...]
```

4.5.3 Roughness and clumping index

Similar to TSEB we need to calculate the aerodynamic roughness parameters, along with the clumping index for each vegetation layer

On the other hand, for the grass understory we can assume that it is horizontally homogeneous and non-clumped so we don't need to calculate the clumping index

4.5.4 Three-Source Net Shortwave Radiation Estimates

For 3SEB, we use an adapted radiation transmission model similar to the model described in Campbell & Norman (1998) and above for TSEB. The only difference is that we simulate radiation transmission for an extra vegetation source. In this case, we also estimate the projected shadow of the overstory onto the substrate system, taking into account its influence on the proportion of direct/diffuse radiation in the substrate.

Note

The details regarding the adapted three-source shortwave radiation transmission model is found in the supplementary information in BurchardLevine et al. (2021) and in the [py3SEB code](#)

See Also

You can take a deeper look at the shortwave net radiation calculation for 3SEB [here](#)

4.5.5 Computing Sensible heat flux with 3SEB

The key distinction of 3SEB is that we need to separate sensible heat fluxes between vegetation overstory, understory and soil layers. All this, using the radiometric land surface temperature (LST) derived from remote sensing as the key boundary condition. It is assumed that the singular LST observation is the combination of signals stemming from all three sources such as :

$$\begin{aligned} LST &= (f_{ov}(\theta)T_{ov}^4 + [1 - f_{ov}(\theta)]T_{sub}^4)^{1/4} \\ T_{sub} &= (f_{un}(\theta)T_{un}^4 + [1 - f_{un}(\theta)]T_s^4)^{1/4} \end{aligned} \quad (4.8)$$

where $f_c(\theta)$ is the fraction of vegetation observed by the sensor (either for understory (un) and overstory (ov)). Since the above equations consists of three unknowns and only two equations, an iterative process is conducted using two initial guesses of potential canopy transpiration for both vegetation sources, and under the assumption that during daytime hours condensation for the soil/substrate should not occur. The initial canopy latent and sensible heat fluxes are estimated based on the PRIESTLEY & TAYLOR (1972) formulation for potential transpiration (Eq. (3)) and Eq. (1) as discussed above for TSEB.

In 3SEB, resistance framework is first treated as a parallel (i.e., uncoupled) tree-substrate system to obtain tree canopy sensible heat flux (H_C) and substrate (understory vegetation+soil) (H_{sub}) using the heat transport equations:

$$\begin{aligned} H_C &= \rho C_p \frac{T_C - T_a}{R_A} \\ H_{sub} &= \rho C_p \frac{T_{sub} - T_a}{R_A + R_{sub}} \end{aligned} \quad (4.9)$$

[illegible]

```

TSEB.res.GRASS),

np.full(lai_un.shape,

f_c=np.full(lai_ov.shape, f_c_un))

# Ensure realistic values
d_0_un[d_0_un < 0] = 0
z_0m_un[np.isnan(z_0m_un)] = z0_soil
z_0m_un[z_0m_un < z0_soil] = z0_soil

# Calculate clumping index for overstory
omega0_ov = TSEB.CI.calc_omega0_Kustas(lai_ov, f_c_ov, x_LAD=x_lad,
isLAIeff=True)
Omega_ov = TSEB.CI.calc_omega_Kustas(omega0_ov, np.minimum(sza, 90),
w_C=w_c_ov)
F_ov = lai_ov/f_c_ov # Real LAI_ov
# effective LAI (tree layer)
lai_ov_eff = F_ov * Omega_ov
print("Parsed clumping and roughness paramters correctly, you can continue to
next cell")

Parsed clumping and roughness paramters correctly, you can continue to next
cell

# setting constant values across image array
# overstory
lai_ov = np.full_like(lst_ar, 0.4) # acquried from LAI decomposition (see
above)
fc_ov_ar = np.full_like(lst_ar, f_c_ov) # average tree fractional cover
hc_ov_ar = np.full_like(lst_ar, h_c_ov) # average tree height

# understory
lai_un = (lai_ar - lai_ov) /(1-f_c_ov)
lai_un[lai_un<0.3] = 0.3
fc_un_ar = np.full_like(lst_ar, f_c_un)# homogeneous grass cover
hc_un_ar = np.full_like(lst_ar, h_c_un) # grass cover height

# get sun angles during S3 overpass datetime
sza, saa = TSEB.met.calc_sun_angles(lat,
lon,
stdlon,
s3_overpass_datetime.dayofyear,
s3_overpass_datetime.hour +

s3_overpass_datetime.minute / 60.)
# local/real LAI of overstory
F_ov = lai_ov/ fc_ov_ar
# clulmping of overstory
omega_0 = TSEB.CI.calc_omega0_Kustas(lai_ov, fc_ov_ar, x_LAD=1)
omega = TSEB.CI.calc_omega_Kustas(omega_0, np.minimum(sza, 90), w_C=w_c)
# effective LAI of overstory
lai_ov_eff = F_ov * omega

## roughness parameters
# Overstory (i.e. Evergreen trees)
[z_0m_ov, d_0_ov] = TSEB.res.calc_roughness(lai_ov,
hc_ov_ar,
np.full_like(lai_ov, w_c),
np.full_like(lai_ov,

TSEB.res.BROADLEAVED_E),

f_c=fc_ov_ar)

# Ensure realistic values
d_0_ov[d_0_ov < 0] = 0
z_0m_ov[np.isnan(z_0m_ov)] = z0_soil
d_0_ov[d_0_ov < z0_soil] = z0_soil

# Understory (i.e. grasses)
# =====

```

```
z_0m_un, d_0_un = TSEB.res.calc_roughness(lai_un,
                                          hc_un_ar,
                                          np.full_like(lai_un, w_c),
                                          np.full_like(lai_un,
TSEB.res.GRASS),
                                          f_c=fc_un_ar)
```

```
# Ensure realistic values
d_0_un[d_0_un < 0] = 0
z_0m_un[np.isnan(z_0m_un)] = z0_soil
z_0m_un[z_0m_un < z0_soil] = z0_soil
print("Parsed biophysical and structural parameters correctly, you can
continue to next cell")
```

Parsed biophysical and structural parameters correctly, you can continue to next cell

4.5.7 Running py3SEB with Satellite inputs

```
print('Estimating net shortwave radiation [....]')
```

```
# Estimates the direct and diffuse solar radiation
difvis, difnir, fvis, fnir = TSEB.rad.calc_diffuse_ratio(np.full_like(lst_ar,
sw_in),
```

```
np.full_like(lst_ar,
```

```
sza),
```

```
press=np.full_like(lst_ar, P))
```

```
par_dir = fvis * (1. - difvis) * np.full_like(lst_ar, sw_in)
```

```
nir_dir = fnir * (1. - difnir) * np.full_like(lst_ar, sw_in)
```

```
par_dif = fvis * difvis * np.full_like(lst_ar, sw_in)
```

```
nir_dif = fnir * difnir * np.full_like(lst_ar, sw_in)
```

```
# net radiation transmission through the three sources
```

```
sn_c_ov, sn_s, sn_c_un = py3seb.calc_Sn_Campbell(
```

```
    lai_ov, # LAI of overstory
```

```
    lai_un, # LAI of understory
```

```
    np.full_like(lst_ar, sza), # sun zenith angle
```

```
    par_dir + nir_dir, # direct incoming irradiance
```

```
    par_dif + nir_dif, # diffuse incoming irradiance
```

```
    fvis, # fration of total visible radiation
```

```
    fnir, # fration of total NIR radiation
```

```
    np.full_like(lst_ar, 0.05), # Overstory Broadband leaf reflectance in the
visible region (400-700nm)
```

```
    np.full_like(lst_ar, 0.05), # Understory Broadband leaf reflectance in the
visible region (400-700nm)
```

```
    np.full_like(lst_ar, 0.08), # Overstory Broadband leaf transmittance in
the visible region (400-700nm)
```

```
    np.full_like(lst_ar, 0.08), # Understory Broadband leaf tansmittance in
the visible region (400-700nm)
```

```
    np.full_like(lst_ar, 0.26), # Overstory Broadband leaf reflectance in the
NIR region (700-2500nm)
```

```
    np.full_like(lst_ar, 0.26), # Understory Broadband leaf reflectance in the
NIR region (700-2500nm)
```

```
    np.full_like(lst_ar, 0.33), # Overstory Broadband leaf transmittance in
the NIR region (700-2500nm)
```

```
    np.full_like(lst_ar, 0.33), # Understory Broadband leaf transmittance in
the NIR region (700-2500nm)
```

```
    np.full_like(lst_ar, 0.07), # Soil Broadband reflectance in the visible
region (400-700nm)
```

```
    np.full_like(lst_ar, 0.28), # Soil Broadband reflectance in the NIR region
(700-2500nm)
```

```
    hc_ov_ar, # overstory canopy height
```

```
    hc_ov_ar * 0.5, # ratio of hc where leaf canopy begins in overstory layer
```

```
    w_c_ov, # width to height ratio of overstory
```

```
    fc_ov_ar, # Fractional cover of overstory
```



```

x_LAD=1, # Overstory x parameter for the ellipsoildal Leaf Angle
Distribution function of Campbell 1988
x_LAD_sub=1, # Understory x parameter for the ellipsoildal Leaf Angle
Distribution function of Campbell 1988
LAI_eff=lai_ov, # Effective LAI of overstory
LAI_eff_sub=lai_un) # Effective LAI of understory

vza = np.full_like(lst_ar, 21.88) # from S3-SLSTR product

print('Running 3SEB [...]')
# using constant ratio appraoch to estimate G
G_constant = 0.35
calcG = [[1], G_constant]

# use Norman and Kustas 1995 resistance framework
Resistance_flag=[0,{}]

[flag_PT_all, T_S, T_C_ov, T_C_un, T_AC, Ln_sub, Ln_C_ov, Ln_C_un, Ln_S,
LE_C_ov,
H_C_ov, LE_C_un, H_C_un, LE_S, H_S, G_mod, R_S, R_sub, R_X, R_A, u_friction,
L,
n_iterations] = py3seb.ThreeSEB_PT_PT_SEQ(
    lst_ar,
    vza,
    np.full_like(lst_ar, Ta),
    np.full_like(lst_ar, u),
    np.full_like(lst_ar, ea),
    np.full_like(lst_ar, P),
    sn_c_ov,
    sn_s,
    sn_c_un,
    np.full_like(lst_ar, lw_in),
    lai_ov,
    lai_un,
    hc_ov_ar,
    hc_un_ar,
    e_v,
    e_v,
    e_s,
    z_0m_ov,
    z_0m_un,
    d_0_ov,
    d_0_un,
    z_u,
    z_t,
    leaf_width=leaf_width_ov,
    leaf_width_sub=leaf_width_un,
    f_c=fc_ov_ar,
    f_c_sub=fc_un_ar,
    f_g=np.full_like(lst_ar, 0.9),
    f_g_sub=np.full_like(lst_ar, 0.75),
    calcG_params=calcG,
    resistance_form=Resistance_flag)

# save ouputs in outdict
LE = LE_C_ov + LE_C_un + LE_S
H = H_C_ov + H_C_un + H_S

Rn_C = Ln_C_ov + sn_c_ov
Rn_C_sub = Ln_C_un + sn_c_un
Rn_S = Ln_S + sn_s
Rn = Rn_C + Rn_C_sub + Rn_S

fig = make_subplots(
    rows=1,
    cols=3,

```



```

horizontal_spacing=0.03,
subplot_titles=("LE", "H", "Rn"))

# LE
fig.add_trace(
    go.Heatmap(
        z=LE,
        colorscale="Blues",
        zmin=0,
        zmax=400,
        colorbar=dict(
            title="LE (W/m2)",
            orientation="h",
            x=0.13,
            y=-0.2,
            len=0.3,
            thickness=15)),
    row=1, col=1)

# H
fig.add_trace(
    go.Heatmap(
        z=H,
        zmin=0,
        zmax=400,
        colorscale="Reds",
        colorbar=dict(
            title="H (W/m2)",
            orientation="h",
            x=0.48,
            y=-0.2,
            len=0.3,
            thickness=15)),
    row=1, col=2)

# Rn
fig.add_trace(
    go.Heatmap(
        z=Rn,
        zmin=0,
        zmax=600,
        colorscale="Oranges",
        colorbar=dict(
            title="Rn (W/m2)",
            orientation="h",
            x=0.82,
            y=-0.2,
            len=0.3,    # colorbar length
            thickness=15)),
    row=1, col=3)

fig.update_layout(
    title="Modelled instantaneous fluxes from 3SEB",
    height=500,
    width=1500)

# Make pixels square like imshow
fig.update_yaxes(autorange="reversed", scaleanchor="x", row=1, col=1)
fig.update_yaxes(autorange="reversed", scaleanchor="x2", row=1, col=2)
fig.update_yaxes(autorange="reversed", scaleanchor="x3", row=1, col=3)

Estimating net shortwave radiation [...]
Running 3SEB [...]
Iteration: 0, non-converged pixels: 9914, max L diff: inf, total time:
0.000029, loop time: 0.000029

```

```

Iteration: 1, non-converged pixels: 9914, max L diff: inf, total time:
0.019246, loop time: 0.019217
Iteration: 2, non-converged pixels: 9914, max L diff: 3.597924, total time:
0.054261, loop time: 0.035015
Iteration: 3, non-converged pixels: 9912, max L diff: 0.469195, total time:
0.087093, loop time: 0.032831

Iteration: 4, non-converged pixels: 9903, max L diff: 0.379073, total time:
0.120238, loop time: 0.033146

Iteration: 5, non-converged pixels: 9840, max L diff: 0.150758, total time:
0.154424, loop time: 0.034186

Iteration: 6, non-converged pixels: 9793, max L diff: 0.140238, total time:
0.187750, loop time: 0.033325
Iteration: 7, non-converged pixels: 9513, max L diff: 0.141839, total time:
0.220652, loop time: 0.032902
Iteration: 8, non-converged pixels: 9433, max L diff: 0.141105, total time:
0.248465, loop time: 0.027813
Iteration: 9, non-converged pixels: 9356, max L diff: 0.011203, total time:
0.270106, loop time: 0.021641
Iteration: 10, non-converged pixels: 9247, max L diff: 0.005794, total time:
0.290248, loop time: 0.020142
Iteration: 11, non-converged pixels: 1388, max L diff: 0.002961, total time:
0.310241, loop time: 0.019993
Iteration: 12, non-converged pixels: 79, max L diff: 0.001522, total time:
0.316973, loop time: 0.006733
Finished iterations with a max. L diff: 0.0007782386965118349

..Finished..

```

4.5.8 Flux partitioning between sources

With 3SEB we have the added value of being able to estimate the partitioning between the two main vegetation components (i.e. trees and grasses)

```

fig = make_subplots(
    rows=1,
    cols=3,
    horizontal_spacing=0.03,
    subplot_titles=("LE_ov", "LE_un", "LE_s"))

# LE
fig.add_trace(
    go.Heatmap(
        z=LE_C_ov,
        colorscale="Blues",
        zmin=0,
        zmax=300,
        colorbar=dict(
            title="LE_ov (W/m²)",
            orientation="h",
            x=0.13,
            y=-0.2,
            len=0.3,
            thickness=15)),
    row=1,col=1)

# H
fig.add_trace(
    go.Heatmap(
        z=LE_C_un,
        zmin=0,
        zmax=300,
        colorscale="Blues",
        colorbar=dict(
            title="LE_un (W/m²)",

```

```

        orientation="h",
        x=0.48,
        y=-0.2,
        len=0.3,
        thickness=15)),
    row=1, col=2)

# Rn
fig.add_trace(
    go.Heatmap(
        z=LE_S,
        zmin=0,
        zmax=300,
        colorscale="Blues",
        colorbar=dict(
            title="LE_s (W/m²)",
            orientation="h",
            x=0.82,
            y=-0.2,
            len=0.3,    # colorbar length
            thickness=15)),
    row=1, col=3)

fig.update_layout(
    title="LE Partitioning from 3SEB",
    height=500,
    width=1500)

# Make pixels square like imshow
fig.update_yaxes(autorange="reversed", scaleanchor="x", row=1, col=1)
fig.update_yaxes(autorange="reversed", scaleanchor="x2", row=1, col=2)
fig.update_yaxes(autorange="reversed", scaleanchor="x3", row=1, col=3)

```

Questions:

- How do the modelled fluxes compare to the tower measurements?
- How could we get a map of daily ET (mm/day) from these instantaneous fluxes?
- Why do we not see so much spatial variability in LE_{ov} ?

4.6 Conclusions

- In this notebook we learn the basics of the low level code of pyTSEB to run TSEB-PT using as input one single value
- We showed an example of running TSEB-PT with spatialized inputs (e.g. satellite-based inputs)
- We also learned the basics of the low level code of py3SEB and its main differences compared to TSEB
- For additional information regarding to derive all inputs required for TSEB/3SEB using Copernicus data from the [Copernicus Data Space Ecosystem](#) see [Sen-ET OpenEO Toolbox](#) and [py3seb-copernicus](#)

5. Calculation of Shortwave and Longwave Net Radiation

5.1 Summary

This interactive Jupyter Notebook has the objective of showing the implementation of TSEB-PT model in the [pyTSEB package](#).

5.2 Instructions

Read carefully all the text and follow the instructions.

Once each section is read, run the jupyter code cell underneath (marked as In []) by clicking the icon Run, or pressing the keys SHIFT+ENTER of your keyboard. A graphical interface will then display, which allows you to interact with and perform the assigned tasks.

To start, please run the following cell to import all the packages required for this notebook. Once you run the cell below, an acknowledgement message, stating all libraries were correctly imported, should be printed on screen.

```
%matplotlib inline
from pathlib import Path
from ipywidgets import interact, interactive, fixed, widgets
from IPython.display import display
import numpy as np
import pandas as pd
import tabulate
from pyTSEB import TSEB
from py3seb import py3seb
import matplotlib.pyplot as plt
from plotly.subplots import make_subplots
import plotly.graph_objects as go
print("Libraries imported!")
```

Libraries imported!

5.3 Net shortwave radiation

To compute the net shortwave radiation and its partitioning between the canopy and the soil we are using the model of Campbell & Norman (1998, Chapter 15). The key aspect of this model is the calculation of the transmitted shortwave radiation through the canopy (T_C)

See Also

The full description of the radiation model can be found in Campbell & Norman (1998, Chapter 15)

The transmittance of shortwave radiation through the canopy depends on the wavelength due to vegetation absorbing a greater portion of the photosynthetically active radiation (PAR, 400–700 nm spectrum) than near-infrared radiation (NIR, 700–2500 nm spectrum) wavelengths. (T_C is partitioned into two components (direct/diffuse) and in two spectral band (PAR/NIR)

Note

We could include more spectral bands, such as (PAR/NIR/SWIR), with a slight increase of computational cost.

The direct-beam spectral (transmittance ($\tau_{C,DIR,\lambda}$) at a given solar zenith angle (θ_S) is calculated following the equations of Campbell & Norman (1998) for a single layer crop:

$$\tau_{C,DIR,\lambda}(\theta_S) = \frac{(\rho_{C,\lambda}^*(\theta_S)^2 - 1) \exp(-\sqrt{\zeta_\lambda} \kappa_b(\theta_S) LAI)}{(\rho_{C,\lambda}^* \rho_{S,\lambda} - 1) + \rho_{C,\lambda}^*(\psi) (\rho_{C,\lambda}^*(\theta_S) - \rho_{S,\lambda}) \exp(-2\sqrt{\zeta_\lambda} \kappa_b(\theta_S) LAI)} \quad (5.1)$$

with (λ) being either the PAR or NIR. $\rho_{C,\lambda}^*(\psi)$ is the beam spectral reflection coefficient for a deep canopy with non-horizontal leaves (see Eq. (2)), ζ_λ is the leaf absorptivity, κ_b is the extinction coefficient for direct-beam radiation (per LAI unit), and $\rho_{S,\lambda}$ is the soil spectral reflectance. The multiple scattering between the soil and the canopy is accounted for in the $\rho_{C,\lambda}^*$ and $\rho_{S,\lambda}$ terms.

$$\rho_{C,\lambda}^*(\theta_S) = \frac{2\kappa_b(\theta_S)\rho_\lambda^H}{\kappa_b(\theta_S) + 1} \quad (5.2)$$

$\rho_\lambda^H = \frac{1-\sqrt{\zeta_\lambda}}{1+\sqrt{\zeta_\lambda}}$ is the reflectance factor for a canopy with horizontal leaves.

Finally, the canopy beam extinction $\kappa_b(\psi)$ is calculated based on the ellipsoidal LIDF of Campbell (1990)

$$\kappa_b(\theta_S) = \frac{\sqrt{\chi^2 + \tan^2 \theta_S}}{\chi + 1.774(\chi + 1.182)^{-0.733}} \quad (5.3)$$

with χ the Campbell (1990) LIDF parameter

Note

$\chi = 1$ is the standard value for LIDF spherical distribution, with values lower than 1 reflecting planophylles canopies and $\chi > 1$ for erectophille canopies.

Diffuse spectral transmittance ($\tau_{C,DIF,\lambda}$) is calculated by numerically integrating κ_b over the hemisphere

$$\kappa_d = 2 \int_0^\pi \kappa_b(\psi) \sin \psi \cos \psi d\psi \quad (5.4)$$

and replacing κ_b by κ_d in Eq. (1)

Similarly the canopy direct spectral albedo is computed as:

$$\rho_{C,DIR,\lambda}(\theta_S) = \frac{\rho_{C,\lambda}^*(\theta_S) + \left[\frac{\rho_{C,\lambda}^*(\theta_S) - \rho_{S,\lambda}}{\rho_{C,\lambda}^*(\theta_S)\rho_{S,\lambda} - 1} \right] \exp(-2\sqrt{\zeta_\lambda}\kappa_b(\theta_S)LAI)}{1 + \rho_{C,\lambda}^*(\theta_S) + \left[\frac{\rho_{C,\lambda}^*(\theta_S) - \rho_{S,\lambda}}{\rho_{C,\lambda}^*(\theta_S)\rho_{S,\lambda} - 1} \right] \exp(-2\sqrt{\zeta_\lambda}\kappa_b(\theta_S)LAI)} \quad (5.5)$$

and the diffuse canopy albedo ($\rho_{C,DIF,\lambda}$) by replacing $\kappa_b(\theta_S)$ by κ_d

Therefore as inputs for computin net shortwave radiation we need to provide as input forcing the solar irradiance, direct and diffuse in the PAR and NIR, and then as main input the LAI , followed by soil and leaf spectra and Campbell (1990) LIDF parameter (χ).

See Also

The code for estimating the diffuse and bean radiation is based on Weiss & Norman (1985) and can be found at the [pyTSEB GitHub repository](#)

The code for running the Campbell & Norman (1998) model can be found at the pyTSEB GitHub repository: [canopy spectral properties](#) and [estimation of net shortwave radiation](#)

5.3.1 Retrieval of broadband spectral properties

For estimating leaf spectra we will use the leaf traits retrievals from the satellite biophysical traits at input/canopy. With the information of these traits we could run in forward mode the ProspectD leaf RTM to get a spectral reflectance and transmittance that could then be integrated to the required broadband regions. However, running ProspectD for a large array of pixels is too computationally expensive. For that reason we have developed a ProspectD emulator (Guzinski et al., 2021): we generated broadband leaf reflectances and transmittances in the PAR and NIR from a large range of ProspectD simulations, covering all

plausible range of leaf traits, which then will be used to train another random forest model that will relate the leaf traits to the broadband PAR and NIR leaf reflectance and transmittance.

Soil reflectance has a smaller role in most situations, since the radiation reaching the ground is smaller due to the interception of light by the canopy. However, in sparse and semi-arid conditions, where vegetation is scarce or even not present, soil albedo plays obviously a significant role. Furthermore, these semi-arid areas are usually characterized by brighter soils since they are composed by sands, salts and with a small fraction of organic matter. For that reason in Guzinski et al. (2021) we developed a method to unmix the broadband soil reflectance ($\rho_{soil,\lambda}$) from the satellite observed reflectances (Eq. \ref{eq:rho-soil})

$$\rho_{soil,\lambda} = \frac{\rho_{surface,\lambda} - [1 - P_0(\theta_v)]\rho_{leaf,\lambda}}{P_0(\theta_v)} \quad (5.6)$$

where $P_0(\theta_v)$ is the canopy gap fraction at the satellite observation angle, computed by the Beer-Lambert law ($\exp[-\kappa(\theta_v)LAI]$). $\rho_{surface,\lambda}$ is the broadband surface reflectance, and $\rho_{leaf,\lambda}$ is the leaf broadband reflectance.

5.3.1.1 Select a site

```
# Set the canopy and eddy covariance folders
input_dir = Path().absolute() / "input"
ec_dir = input_dir / "eddy_covariance"
sites = ec_dir.glob("*SUBSET_HH.csv")
sites = [i.stem.split("_")[1] for i in sites]
# Site metadata file
site_metadata_file = input_dir / "sites.csv"
site_metadata = pd.read_csv(site_metadata_file, sep=";")
valid = np.isin(site_metadata["SITE_ID"], sites)
site_metadata = site_metadata.loc[valid]

w_site_list = widgets.Dropdown(
    options=sites,
    value=sites[0],
    description='Select site:',
    rows=9
)
display(w_site_list)
site_metadata

Dropdown(description='Select site:', options=('ES-Abr', 'ES-LMa', 'ES-HdD',
'ES-LM2', 'ES-Gdn', 'ES-Cnd', 'ES-...
```

	SITE_ID	IGBP	LOCATION_LAT	LOCATION_LONG	LOCATION_ELEV	UTC_OFFSET
100	ES-Abr	SAV	38.701840	-6.785881	279.0	1.0
104	ES-Cnd	WSA	37.914997	-3.227659	370.0	1.0
105	ES-Crg	CSH	39.119260	-0.958520	1040.0	1.0
108	ES-Gdn	EBF	37.910915	-3.228444	370.0	1.0
109	ES-Hen	ENF	38.411087	-1.697534	425.0	1.0
111	ES-LM1	SAV	39.942690	-5.778683	265.0	1.0
112	ES-LM2	SAV	39.934593	-5.775881	265.0	1.0
113	ES-LMa	SAV	39.940330	-5.774650	265.0	1.0
117	ES-Mcd	EBF	39.588190	-0.399690	61.0	1.0
120	ES-Pzb	GRA	38.362118	-4.767329	680.0	1.0
122	ES-TzM	CRO	39.265556	-1.939722	727.0	1.0
123	ES-VDA	GRA	42.151110	1.447380	1767.0	1.0

5.3.1.2 Read the LAI and Micrometeorology data

We will read and merge the [input hourly micrometeorology](#) and [input biophysical traits](#) ASCII files.

See Also

A detailed description of the Eddy Covariance measurements and the FLUXNET/OneFLUX post- processing is described by Pastorello et al. (2020).

Estimates of daily LAI were obtained from an hybrid inversion of PROSPECTD and 4SAIL radiative transfer models using multispectral satellite data from Sentinel-2 and Landsat. For more details about the approach you can read the notebook

[301_spectral_properties](#)

```
bio_dir = input_dir / "canopy"

# Get site properties
site = w_site_list.value
site_metadata = pd.read_csv(site_metadata_file, sep=";")
site_id = site_metadata["SITE_ID"] == site
lat = site_metadata.loc[site_id, "LOCATION_LAT"].item()
lon = site_metadata.loc[site_id, "LOCATION_LONG"].item()
elev = site_metadata.loc[site_id, "LOCATION_ELEV"].item()
utc_offset = site_metadata.loc[site_id, "UTC_OFFSET"].item()
igbp = site_metadata.loc[site_id, "IGBP"].item()

# Set the input files based on the chosen site
bio_filename = bio_dir / f"{site}_HLS-l2c.csv"
ec_filename = ec_dir / f"FLX_{site}_FLUXNET_SUBSET_HH.csv"
print(f"Biophysical traits file path is {bio_filename}")
print(f"EC file path is {ec_filename}")

# Read the fluxnet EC table and preprocess
ec = pd.read_csv(ec_filename, sep=',', na_values=-9999)
ec["TIMESTAMP_START"] = pd.to_datetime(
    ec["TIMESTAMP_START"], format="%Y%m%d%H%M")

ec["TIMESTAMP_END"] = pd.to_datetime(
    ec["TIMESTAMP_END"], format="%Y%m%d%H%M")

new_cols = {}
new_cols["TIMESTAMP"] = pd.to_datetime(
    ec[["TIMESTAMP_START", "TIMESTAMP_END"]].mean(axis=1))

new_cols["DATE"] = new_cols["TIMESTAMP"].dt.date
ec = pd.concat([ec, pd.DataFrame(new_cols)], axis=1)

# Read the satellite biophysical traits table
bio = pd.read_csv(bio_filename, sep=";", na_values=-9999)
bio["DATE"] = pd.to_datetime(bio["DATE"], format="%Y-%m-%d").dt.date

# Merge both tables by date
ec = ec.merge(bio, on="DATE")

fig = make_subplots(rows=2, cols=1,
                    shared_xaxes=True,
                    horizontal_spacing=0.01,
                    subplot_titles=("LAI", "Leaf pigments"))

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI"],
                           name="LAI", mode="lines", line={"color": "black"}),
              row=1, col=1)

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI"] * ec["FG"],
```



```

name="gLAI", mode="lines", line={"color": "green",
"dash": "dash"}),
    row=1, col=1)

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["ANT"],
    name="Leaf Antocyanins", mode="lines", line={"color":
"red"}),
    row=2, col=1)

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["CAB"],
    name="Leaf Chlorophyll", mode="lines", line={"color":
"green"}),
    row=2, col=1)

fig.update_xaxes(title_text="Date", row=3, col=1)
fig.update_layout(title_text=f"Biophysical traits timeseries in {site}")

Biophysical traits file path is /home/hectornieto/codes/courses/curso-aet/
input/canopy/ES-Abr_HLS-l2c.csv
EC file path is /home/hectornieto/codes/courses/curso-aet/input/
eddy_covariance/FLX_ES-Abr_FLUXNET_SUBSET_HH.csv

```

5.3.1.3 Get the broadband spectral properties

5.3.1.4 Estimate net shortwave radiation based on biophysical traits and incoming irradiance

Now we can execute the cell below to run the Campbell & Norman (1998) radiative transfer model implemented in pyTSEB using the site selected in [Select a site](#) section.

Besides of LAI, its leaf inclination distribution function and the spectral properties, in TSEB we also make use of the fractional cover (f_c) and canopy shape (w_c/h_c) for clumped canopies. These are used to compute a clumping index that converts the LAI to effective values based on the solar incidence angle (since in clumped canopies shading can play a significant role)

Warning

Both f_c and w_c/h_c are structural variables in TSEB and they cannot be operationally retrieved from satellite data. The typical FCOVER product **should not** be used as surrogate for f_c since FCOVER not only represents the canopy clumpiness but also the canopy gap fraction.

In operational implementations, we derived a static f_c value only for clumped canopies using prescribed values based on land cover/plant functional type

```

RF_OBJECT = input_dir / "leaf_spectra.joblib"

def leaf_spectra(cab, car, cbrown, cw, cm, ant,
    reg_object_file=RF_OBJECT):
    from joblib import load
    """Estimates broadleaf leaf spectra from PROSPECT-D biophysical traits"""

    reg_object = load(reg_object_file)
    x_array = pd.DataFrame({"Cab": cab,
                            "Car": car,
                            "Cbrown": cbrown,
                            "Cw": cw,
                            "Cm": cm,
                            "Ant": ant})

    abs_est = reg_object.predict(x_array).T
    return abs_est

```

```
def components_spectra(input_df, lai_sparse=1e6):
    [input_df["RHO_LEAF_VIS"], input_df["RHO_LEAF_NIR"],
     input_df["TAU_LEAF_VIS"], input_df["TAU_LEAF_NIR"]] = leaf_spectra(
        input_df["CAB"].values,
        input_df["CAR"].values,
        input_df["CBROWN"].values,
        input_df["CW"].values,
        input_df["CM"].values,
        input_df["ANT"].values)

    input_df["RHO_SOIL_VIS"], input_df["RHO_SOIL_NIR"] = soil_spectra(
        input_df, lai_sparse=lai_sparse)

    return input_df

def soil_spectra(input_df, lai_sparse):
    """Unmix the broadband surface reflectance between
    PROSPECT-D leaf reflectance and estimated soil spectra
    """

    rho_soil_vis = np.full_like(input_df["LAI"], 0.15)
    rho_soil_nir = np.full_like(input_df["LAI"], 0.25)

    # Bare soil conditions, soil reflectance = satellite reflectance
    bare = np.logical_or(input_df["LAI"] == 0, input_df["F_C"] == 0)

    rho_soil_vis[bare] = input_df.loc[bare, "RHO_VIS"]
    rho_soil_nir[bare] = input_df.loc[bare, "RHO_NIR"]

    # Sparse vegetation conditions
    sparse = np.logical_and.reduce((input_df["LAI"] > 0,
                                    input_df["LAI"] <= lai_sparse,
                                    input_df["F_C"] > 0))

    local_lai = input_df.loc[sparse, "LAI"] / input_df.loc[sparse, "F_C"]
    fvc = TSEB.calc_F_theta_campbell(0,
                                     local_lai,
                                     x_LAD=input_df.loc[sparse, "X_LAD"])
    f_soil = (1 - input_df.loc[sparse, "F_C"]) + input_df.loc[sparse, "F_C"] *
    (1.0 - fvc)

    rho_soil_vis[sparse] = (input_df.loc[sparse, "RHO_VIS"] - (1 - f_soil)
                           * input_df.loc[sparse, "RHO_LEAF_VIS"]) / f_soil
    rho_soil_nir[sparse] = (input_df.loc[sparse, "RHO_NIR"] - (1 - f_soil)
                           * input_df.loc[sparse, "RHO_LEAF_NIR"]) / f_soil

    rho_soil_vis = np.clip(rho_soil_vis, 0.01, 0.99)
    rho_soil_nir = np.clip(rho_soil_nir, 0.01, 0.99)
    return rho_soil_vis, rho_soil_nir

print("Leaf and soil spectra functions correctly parsed, you can continue")
Leaf and soil spectra functions correctly parsed, you can continue

import yaml

# Set the input TSEB ancillary parameters
tseb_param_file = input_dir / "TSEB_params.yaml"

# Read metadata and get site info
site_metadata = pd.read_csv(site_metadata_file, sep=";")
site_id = site_metadata["SITE_ID"] == site

# Avoid pandas fragmentation warnings
ec = ec.copy()
```

```
# Compute the Campbell chi LIDF parameter from Campbell mean leaf angle
ec.loc[:, "X_LAD"] = TSEB.rad.leafangle_2_chi(ec["LEAF_ANGLE"].values)

with open(tseb_param_file, "r") as fid:
    tseb_params = yaml.load(fid, Loader=yaml.FullLoader)

ec.loc[:, "F_C"] = tseb_params["F_C"][igbp]
w_c = tseb_params["W_C"][igbp]

# The ASCII table is missing the solar angles, so we will use the
# calc_sun_angles function of TSEB to compute the angles based on site location
# and timestamp
# The standard meridian time zone (15 deg. per hour)
stdlon = 15 * utc_offset
theta, saa = TSEB.met.calc_sun_angles(
    np.full_like(ec["LAI"].values, lat),
    np.full_like(ec["LAI"].values, lon),
    np.full_like(ec["LAI"].values, stdlon),
    ec['TIMESTAMP'].dt.dayofyear.values,
    ec['TIMESTAMP'].dt.hour.values + ec['TIMESTAMP'].dt.minute.values / 60.)

F = ec["LAI"].values / ec["F_C"].values
omega_0 = TSEB.CI.calc_omega0_Kustas(
    ec["LAI"].values, ec["F_C"].values, x_LAD=ec["X_LAD"].values)

omega = TSEB.CI.calc_omega_Kustas(
    omega_0, np.minimum(theta, 90), w_C=w_c)

lai_eff = F * omega

# Estimate the leaf and soil spectra
ec = components_spectra(ec)
# We append the VIS and PAR spectrum to be computationally more efficient in
# Numpy
rho_leaf = np.array((ec["RHO_LEAF_VIS"].values, ec["RHO_LEAF_NIR"].values))
tau_leaf = np.array((ec["TAU_LEAF_VIS"].values, ec["TAU_LEAF_NIR"].values))
rho_soil = np.array((ec["RHO_SOIL_VIS"].values, ec["RHO_SOIL_NIR"].values))

# In order to estimate Estimates the direct and diffuse solar radiation based
# on the Weiss and Norman model
difvis, difnir, fvis, fnir = TSEB.rad.calc_diffuse_ratio(ec["SW_IN_F"].values,
    theta,

press=np.full_like(theta, 1013.15))
par_dir = fvis * (1. - difvis) * ec["SW_IN_F"].values
nir_dir = fnir * (1. - difnir) * ec["SW_IN_F"].values
par_dif = fvis * difvis * ec["SW_IN_F"].values
nir_dif = fnir * difnir * ec["SW_IN_F"].values

# calculate absorptivity
amean = 1.0 - rho_leaf - tau_leaf
amean_sqrt = np.sqrt(amean)

# D I F F U S E   C O M P O N E N T S
# Integrate to get the diffuse transmittance

taud = 0
for angle in range(0, 90, 5):
    angle = np.radians(angle)
    akd = (np.sqrt(ec["X_LAD"].values**2 + np.tan(theta)**2) \
        / (ec["X_LAD"].values + 1.774 * (ec["X_LAD"].values +
```

```

1.182)**-0.733)) # Eq. 15.4
    taub = np.exp(-akd * ec["LAI"].values)
    taud += taub * np.cos(angle) * np.sin(angle) * np.radians(5)

taud = 2.0 * taud

# Diffuse light canopy reflection coefficients for a deep canopy
akd = -np.log(taud) / ec["LAI"].values
rcpy = (1.0 - amean_sqrt) / (1.0 + amean_sqrt) # Eq 15.7
rdcpy = 2.0 * akd * rcpy / (akd + 1.0) # Eq 15.8

# Diffuse canopy transmission and albedo coeff for a generic canopy (visible)
expfac = amean_sqrt * akd * ec["LAI"].values
neg_exp, d_neg_exp = np.exp(-expfac), np.exp(-2.0 * expfac)
xnum = (rdcpy * rdcpy - 1.0) * neg_exp
xden = (rdcpy * rho_soil - 1.0) + rdcpy * (rdcpy - rho_soil) * d_neg_exp
taudt = xnum / xden # Eq 15.11
fact = ((rdcpy - rho_soil) / (rdcpy * rho_soil - 1.0)) * d_neg_exp
albd = (rdcpy + fact) / (1.0 + rdcpy * fact) # Eq 15.9

# B E A M   C O M P O N E N T S
# Calculate canopy beam extinction coefficient
# Direct beam extinction coeff (spher. LAD)
akb = (np.sqrt(ec["X_LAD"].values**2 + np.tan(theta)**2) \
        / (ec["X_LAD"].values + 1.774 * (ec["X_LAD"].values + 1.182)**-0.733))
# Eq. 15.4

# Direct beam canopy reflection coefficients for a deep canopy
rbcpy = 2.0 * akb * rcpy / (akb + 1.0) # Eq 15.8
# Beam canopy transmission and albedo coeff for a generic canopy (visible)
expfac = amean_sqrt * akb * lai_eff
neg_exp, d_neg_exp = np.exp(-expfac), np.exp(-2.0 * expfac)
xnum = (rbcpy * rbcpy - 1.0) * neg_exp
xden = (rbcpy * rho_soil - 1.0) + rbcpy * (rbcpy - rho_soil) * d_neg_exp
taubt = xnum / xden # Eq 15.11
fact = ((rbcpy - rho_soil) / (rbcpy * rho_soil - 1.0)) * d_neg_exp
albb = (rbcpy + fact) / (1.0 + rbcpy * fact) # Eq 15.9

taubt, taudt, albb, albd, rho_soil = map(np.array,
                                         [taubt, taudt, albb, albd, rho_soil])

taubt[np.isnan(taubt)] = 1
taudt[np.isnan(taudt)] = 1
albb[np.isnan(albb)] = rho_soil[np.isnan(albb)]
albd[np.isnan(albd)] = rho_soil[np.isnan(albd)]

# Compute the canopy and soil net radiation using Campbell RTM
sn_c = ((1.0 - taubt[0]) * (1.0 - albb[0]) * par_dir
        + (1.0 - taubt[1]) * (1.0 - albb[1]) * nir_dir
        + (1.0 - taudt[0]) * (1.0 - albd[0]) * par_dif
        + (1.0 - taudt[1]) * (1.0 - albd[1]) * nir_dif)

sn_s = (taubt[0] * (1.0 - ec["RHO_SOIL_VIS"].values) * par_dir
        + taubt[1] * (1.0 - ec["RHO_SOIL_NIR"].values) * nir_dir
        + taudt[0] * (1.0 - ec["RHO_SOIL_VIS"].values) * par_dif
        + taudt[1] * (1.0 - ec["RHO_SOIL_NIR"].values) * nir_dif)

print("Finished the computation of net shortwave radiation")
Finished the computation of net shortwave radiation

```

5.3.1.5 Evaluate net shortwave radiation

And by running the cell below we validate those estimates with the hourly measurements in the eddy covariance flux tower

```

from model_evaluation import double_collocation as dc
import tabulate

# Avoid pandas fragmentation warnings
ec = ec.copy()

daytime = ec["SW_IN_F"] > 100

sn_model = sn_c + sn_s
sn_obs = ec["SW_IN_F"].values - ec["SW_OUT"].values

bias, mae, rmse = dc.error_metrics(
    sn_obs[daytime], sn_model[daytime])
cor, *_ , d = dc.agreement_metrics(
    sn_obs[daytime], sn_model[daytime])
n, mean_obs, mean_pre, std_obs, std_pre = dc.descriptive_stats(
    sn_obs[daytime], sn_model[daytime])

error_df = {"N": [], "Obs.": [],
            "RMSE": [], "bias": [],
            "scale": [], "r": [], "d": [],
            }
error_df["N"].append(int(n))
error_df["Obs."].append(np.round(mean_obs, 2))
error_df["RMSE"].append(np.round(rmse, 2))
error_df["bias"].append(np.round(bias, 2))
error_df["scale"].append(np.round(std_obs / std_pre, 2))
error_df["r"].append(np.round(cor, 2))
error_df["d"].append(np.round(d, 2))
error_df = pd.DataFrame(error_df, index=["Sn"])

display(error_df)
fig = go.Figure()
fig.add_trace(go.Scattergl(x=sn_model[daytime], y=sn_obs[daytime],
                           name="Shortwave net radiation", mode="markers"))
fig.add_trace(go.Scatter(x=[0, 1000], y=[0, 1000], mode="lines", name="1:1
line", line={"color": "black", "dash": "dash"}))
fig.update_layout(title_text=f"Observed vs. Estimated net radiation at
{site}",
                   yaxis_range=[0, 1000], xaxis_range=[0, 1000],
                   xaxis_title="Estimated (W m-2)", yaxis_title="Observed (W
m-2)")

fig = go.Figure()
fig.add_trace(go.Scatter(x=sn_model[daytime], y=sn_obs[daytime],
                           name="Sn", mode="markers"))
fig.add_trace(go.Scatter(x=[0, 1200], y=[0, 1200], mode="lines", name="1:1
line", line={"color": "black", "dash": "dash"}))
fig.update_layout(title_text=f"Observed vs. Estimated net shortwave radiation
at {site}",
                   yaxis_range=[0, 1200], xaxis_range=[0, 1200],
                   xaxis_title="Estimated (W m-2)", yaxis_title="Observed (W
m-2)")

```

	N	Obs.	RMSE	bias	scale	r	d
Sn	36634	402.91	25.17	0.62	1.05	0.99	1.0

5.4 Net longwave radiation

The net longwave radiation a priori is much simpler as we assume that all emitted and indident longwave radiation is diffuse. In pyTSEB we follow a similar approach as the one described in Kustas & Norman (1999) for partially vegetated surfaces, which also partitions net longwave radiation into the soil ($L_{n,S}$) and the canopy components ($L_{n,C}$). The only

difference between pyTSEB and the formulation of (Kustas & Norman, 1999) is that we incorporate longwave scattering between the soil and the canopy:

$$\begin{aligned} L_{n,S} &= \varepsilon_S \tau_{LAI} L^{down} + \varepsilon_S (1.0 - \tau_{LAI} L^{down}) L_C - L_S \\ L_{n,C} &= \varepsilon_C (1.0 - \tau_{LAI} L^{down}) (L_{sky} + L_S) - 2.0 (1.0 - \tau_{LAI}) L_C \end{aligned} \quad (5.7)$$

where L^{down} is the downwelling atmospheric longwave irradiance, L_S and L_C are the longwave emission of respectively soil and canopy layers, τ_{LAI} is analogous to the shortwave diffuse transmittance and thus it can be calculated from Eq. (4), ε_S is soil emissivity and ε_C can be derived from Eq. (5) ($\varepsilon_C = 1 - \rho_{C,DIF,l}$) and leaf emissivity.

Note

Even we use a modified version of Kustas & Norman (1999), the pyTSEB package also includes the original formulation [here](#)

Attention

However, L_S and L_C depend respectively on soil and canopy temperature, which at this stage is an unknown. For that reason **the application of this equation is done internally when deriving the component temperature and sensible heat fluxes**

```
from pyTSEB.net_radiation import calc_spectra_Campbell

def calc_L_n_Campbell(T_C, T_S, L_dn, lai, emisVeg, emisGrd, x_LAD=1):
    ''' Net longwave radiation for soil and canopy layers

    Estimates the net longwave radiation for soil and canopy layers using
    based on equation 2a
    from [Kustas1999]_ and incorporated the effect of the Leaf Angle
    Distribution based on [Campbell1998]_

    Parameters
    -----
    T_C : float
        Canopy temperature (K).
    T_S : float
        Soil temperature (K).
    L_dn : float
        Downwelling atmospheric longwave radiation (W m-2).
    lai : float
        Effective Leaf (Plant) Area Index.
    emisVeg : float
        Broadband emissivity of vegetation cover.
    emisGrd : float
        Broadband emissivity of soil.
    x_LAD: float, optional
        x parameter for the ellipsoidal Leaf Angle Distribution function,
        use x_LAD=1 for a spherical LAD.

    Returns
    -----
    L_nC : float
        Net longwave radiation of canopy (W m-2).
    L_nS : float
        Net longwave radiation of soil (W m-2).

    References
    -----
    .. [Kustas1999] Kustas and Norman (1999) Evaluation of soil and vegetation
    heat
        flux predictions using a simple two-source model with radiometric
    temperatures for
```

partial canopy cover, Agricultural and Forest Meteorology, Volume 94, Issue 1, Pages 13-29, [http://dx.doi.org/10.1016/S0168-1923\(99\)00005-2](http://dx.doi.org/10.1016/S0168-1923(99)00005-2).

```

'''
def calc_stephan_boltzmann(t_k):
    '''Calculates the total energy radiated by a blackbody.

    Parameters
    -----
    t_k : float
        body temperature (Kelvin)

    Returns
    -----
    M : float
        Emitted radiance (W m-2)'''
    # Stephan Boltzmann constant (W m-2 K-4)
    sb = 5.670373e-8
    m = sb * t_k**4
    return m

# calculate long wave emissions from canopy, soil and sky
L_C = emisVeg * calc_stephan_boltzmann(T_C)
L_S = emisGrd * calc_stephan_boltzmann(T_S)
# Calculate the canopy spectral properties
_, albl, _, taudl = calc_spectra_Campbell(lai,
                                          0,
                                          1.0 - emisVeg,
                                          0,
                                          1.0 - emisGrd,
                                          x_lad=x_LAD,
                                          lai_eff=None)

# calculate net longwave radiation divergence of the soil
L_nS = emisGrd * taudl * L_dn + emisGrd * (1.0 - taudl) * L_C - L_S
L_nC = (1 - albl) * (1.0 - taudl) * (L_dn + L_S) - 2.0 * (1.0 - taudl) *
L_C
return L_nC, L_nS

print("Parsed net longwave radiatin function correctly, you can continue to
next cell")

```

Parsed net longwave radiatin function correctly, you can continue to next cell

5.5 Net shortwave radiation in 3SEB

In this case we will work with the “dehesa” of Majadas de Tiétar

```

site = "ES-LMa"

# fractional cover
f_c_ov = 0.2 # overstory fractional cover (e.g., clumped trees)
f_c_un = 1. # understory fractional cover (e.g., homogeneous grass)
# canopy height
h_c_ov = 8.0 # Canopy height (m) of overstory (e.g. trees)
h_c_un = 0.25

# Thermal spectra
e_c_ov = 0.99 # Overstory Leaf emissivity
e_c_un = 0.97 # Understory Leaf emissivity
e_s = 0.94 # Soil emissivity

```

5.5.1 Decompose LAI between the overstory and understory

```

# Set the input files based on the chosen site
bio_filename = bio_dir / f"{site}_HLS-l2c.csv"
ec_filename = ec_dir / f"FLX_{site}_FLUXNET_SUBSET_HH.csv"

```

```

print(f"Biophysical traits file path is {bio_filename}")
print(f"EC file path is {ec_filename}")

# Read the fluxnet EC table and preprocess
ec = pd.read_csv(ec_filename, sep=',', na_values=-9999)
ec["TIMESTAMP_START"] = pd.to_datetime(
    ec["TIMESTAMP_START"], format="%Y%m%d%H%M")

ec["TIMESTAMP_END"] = pd.to_datetime(
    ec["TIMESTAMP_END"], format="%Y%m%d%H%M")

new_cols = {}
new_cols["TIMESTAMP"] = pd.to_datetime(
    ec[["TIMESTAMP_START", "TIMESTAMP_END"]].mean(axis=1))

new_cols["DATE"] = new_cols["TIMESTAMP"].dt.date
ec = pd.concat([ec, pd.DataFrame(new_cols)], axis=1)

# Read the satellite biophysical traits table
bio = pd.read_csv(bio_filename, sep=";", na_values=-9999)
bio["DATE"] = pd.to_datetime(bio["DATE"], format="%Y-%m-%d").dt.date

# Merge both tables by date
ec = ec.merge(bio, on="DATE")

# Ecosystem LAI
lai_eco = ec['LAI'].values

# Compute the Campbell chi LIDF parameter from Campbell mean leaf angle
ec.loc[:, "X_LAD"] = TSEB.rad.leafangle_2_chi(ec["LEAF_ANGLE"].values)

# Set overstory fraction
ec.loc[:, "F_C"] = f_c_ov

# initialize overstory LAI_ov
lai_ov = np.zeros_like(lai_eco)

years = pd.to_datetime(ec['DATE'].values).year

# get lai_ov for each year
for year in np.unique(years):
    lai_year = ec['LAI'].values[years == year]
    # lowest 10% percentile as estimate of LAI_ov (during summer period)
    lai_min = np.percentile(lai_year, 10)
    lai_ov[years == year] = lai_min

# overstory LAI
ec['LAI_ov'] = lai_ov
# understory LAI
lai_un = (ec['LAI'].values - ec['LAI_ov'].values)/(1-f_c_ov) # (account for
LAI_un below tree canopy)
lai_un[lai_un<0.3] = 0.3
ec['LAI_un'] = lai_un

print('Plotting LAI decomposition time series [...]')
fig = make_subplots(rows=1, cols=1,
                    shared_xaxes=True,
                    horizontal_spacing=0.01,
                    subplot_titles=("LAI", "Leaf pigments"))

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI"],
                           name="LAI_eco", mode="lines", line={"color":
"black"}),
              row=1, col=1)

```



```
fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI_ov"],
                           name="LAI_ov", mode="lines", line={"color": "green",
"dash": "dash"})),
              row=1, col=1)

fig.add_trace(go.Scattergl(x=ec["DATE"], y=ec["LAI_un"],
                           name="LAI_un", mode="lines", line={"color":
"indianred", "dash": "dash"})),
              row=1, col=1)
```

```
fig.update_xaxes(title_text="Date", row=3, col=1)
fig.update_layout(title_text=f"Biophysical traits timeseries in {site}")

Biophysical traits file path is /home/hectornieto/codes/courses/curso-aet/
input/canopy/ES-LMa_HLS-l2c.csv
EC file path is /home/hectornieto/codes/courses/curso-aet/input/
eddy_covariance/FLX_ES-LMa_FLUXNET_SUBSET_HH.csv
```

Plotting LAI decomposition time series [...]

5.5.2 Calculate shortwave net radiation partitioning

```
# The ASCII table is missing the solar angles, so we will use the
calc_sun_angles function of TSEB to compute the angles based on site location
and timestamp
# The standard meridian time zone (15 deg. per hour)
stdlon = 15 * utc_offset
theta, saa = TSEB.met.calc_sun_angles(
    np.full_like(ec['LAI'].values, lat),
    np.full_like(ec["LAI"].values, lon),
    np.full_like(ec["LAI"].values, stdlon),
    ec['TIMESTAMP'].dt.dayofyear.values,
    ec['TIMESTAMP'].dt.hour.values + ec['TIMESTAMP'].dt.minute.values / 60.)

# Calculate clumping index for overstory
omega0_ov = TSEB.CI.calc_omega0_Kustas(lai_ov, f_c_ov, x_LAD=1, isLAIeff=True)
Omega_ov = TSEB.CI.calc_omega_Kustas(omega0_ov, np.minimum(theta, 90),
w_C=w_c)
F_ov = lai_ov/f_c_ov # Real LAI_ov
# effective LAI (tree layer)
lai_ov_eff = F_ov * Omega_ov

# Estimates the direct and diffuse solar radiation
difvis, difnir, fvis, fnir = TSEB.rad.calc_diffuse_ratio(ec["SW_IN_F"].values,
theta,

press=np.full_like(theta, 1013.15))
par_dir = fvis * (1. - difvis) * ec["SW_IN_F"].values
nir_dir = fnir * (1. - difnir) * ec["SW_IN_F"].values
par_dif = fvis * difvis * ec["SW_IN_F"].values
nir_dif = fnir * difnir * ec["SW_IN_F"].values

# Estimate the leaf and soil spectra
ec = components_spectra(ec)

# We append the VIS and PAR spectrum to be computationally more efficient in
Numpy
rho_leaf = np.array((ec["RHO_LEAF_VIS"].values, ec["RHO_LEAF_NIR"].values))
tau_leaf = np.array((ec["TAU_LEAF_VIS"].values, ec["TAU_LEAF_NIR"].values))
rho_soil = np.array((ec["RHO_SOIL_VIS"].values, ec["RHO_SOIL_NIR"].values))

# net radiation transmission through the three sources
sn_c_ov, sn_s, sn_c_un = py3seb.calc_Sn_Campbell(ec['LAI_ov'].values, # LAI of
overstory
ec['LAI_un'].values, # LAI of understory
theta, # sun
zenigth angle
```

```

nir_dir, # direct incoming irradiance
nir_dif, # diffuse incoming irradiance
of total visible radiation
of total NIR radiation
ec["RHO_LEAF_VIS"].values, # Overstory Broadband leaf reflectance in the
visible region (400-700nm)
ec["RHO_LEAF_VIS"].values, # Understory Broadband leaf reflectance in the
visible region (400-700nm)
ec["TAU_LEAF_VIS"].values, # Overstory Broadband leaf transmittance in the
visible region (400-700nm)
ec["TAU_LEAF_VIS"].values, # Understory Broadband leaf transmittance in the
visible region (400-700nm)
ec["RHO_LEAF_NIR"].values, # Overstory Broadband leaf reflectance in the NIR
region (700-2500nm)
ec["RHO_LEAF_NIR"].values, # Understory Broadband leaf reflectance in the NIR
region (700-2500nm)
ec["TAU_LEAF_NIR"].values, # Overstory Broadband leaf transmittance in the NIR
region (700-2500nm)
ec["TAU_LEAF_NIR"].values, # Understory Broadband leaf transmittance in the
NIR region (700-2500nm)
ec["RHO_SOIL_VIS"].values, # Soil Broadband reflectance in the visible region
(400-700nm)
ec["RHO_SOIL_NIR"].values, # Soil Broadband reflectance in the NIR region
(700-2500nm)

overstory canopy height
ratio of hc where leaf canopy begins in overstory layer
height ratio of overstory
Fractional cover of overstory

Overstory x parameter for the ellipsoildal Leaf Angle Distribution function of
Campbell 1988

Understory x parameter for the ellipsoildal Leaf Angle Distribution function
of Campbell 1988
LAI_eff=lai_ov_eff, # Effective LAI of overstory
LAI_eff_sub=ec['LAI_un'].values) # Effective LAI of understory

sn_c_ov[~np.isfinite(sn_c_ov)] = 0
sn_s[~np.isfinite(sn_s)] = 0
sn_c_un[~np.isfinite(sn_c_un)] = 0

# evaluate against tower measurements
daytime = ec["SW_IN_F"] > 100

sn_model = sn_c_ov + sn_c_un + sn_s
sn_obs = ec["SW_IN_F"].values - ec["SW_OUT"].values

bias, mae, rmse = dc.error_metrics(
    sn_obs[daytime], sn_model[daytime])
cor, *, d = dc.agreement_metrics(
    sn_obs[daytime], sn_model[daytime])
n, mean_obs, mean_pre, std_obs, std_pre = dc.descriptive_stats(

```

```

sn_obs[daytime], sn_model[daytime])

error_df = {"N": [], "Obs.": [],
            "RMSE": [], "bias": [],
            "scale": [], "r": [], "d": [],
            }
error_df["N"].append(int(n))
error_df["Obs."].append(np.round(mean_obs, 2))
error_df["RMSE"].append(np.round(rmse, 2))
error_df["bias"].append(np.round(bias, 2))
error_df["scale"].append(np.round(std_obs / std_pre, 2))
error_df["r"].append(np.round(cor, 2))
error_df["d"].append(np.round(d, 2))
error_df = pd.DataFrame(error_df, index=["Sn"])

display(error_df)
print('Plotting modelled shortwave irradiance evaluation [...]')

fig = go.Figure()
fig.add_trace(go.Scattergl(x=sn_model[daytime], y=sn_obs[daytime],
                           name="Shortwave net radiation", mode="markers"))
fig.add_trace(go.Scatter(x=[0, 1000], y=[0, 1000], mode="lines", name="1:1
line",
                        line={"color": "black", "dash": "dash"}))
fig.update_layout(title_text=f"Observed vs. Estimated net radiation at
{site}",
                  yaxis_range=[0, 1000], xaxis_range=[0, 1000],
                  xaxis_title="Estimated (W m-2)", yaxis_title="Observed (W
m-2)")

fig = go.Figure()
fig.add_trace(go.Scatter(x=sn_model[daytime], y=sn_obs[daytime],
                           name="Sn", mode="markers"))
fig.add_trace(go.Scatter(x=[0, 1200], y=[0, 1200], mode="lines", name="1:1
line",
                        line={"color": "black", "dash": "dash"}))
fig.update_layout(title_text=f"Observed vs. Estimated net shortwave radiation
from 3-source model at {site}",
                  yaxis_range=[0, 1200], xaxis_range=[0, 1200],
                  xaxis_title="Estimated (W m-2)", yaxis_title="Observed (W
m-2)")

```

	N	Obs.	RMSE	bias	scale	r	d
Sn	34320	421.56	23.19	2.82	1.05	1.0	1.0

Plotting modelled shortwave irradiance evaluation [...]

5.6 Conclusions

- In this notebook we learn the basics of the low level code of pyTSEB to run TSEB-PT using as input one single value
- A very similar approach would be done for running TSEB-PT with an image or a tabulated timeseries.
- The pyTSEB package includes Jupyter Notebooks that allow using a Graphical User Interface to parse all TSEB inputs
- In an upcoming session we will show you how to derive all inputs required for TSEB using Copernicus data from the [Copernicus Data Space Ecosystem](#)

References

- (2005,). American Society of Civil Engineers. <https://doi.org/10.1061/9780784408056.fm>
- Alfieri, J. G., Kustas, W. P., Nieto, H., Prueger, J. H., Hipps, L. E., McKee, L. G., Gao, F., & Los, S. (2018). Influence of wind direction on the surface roughness of vineyards. *Irrigation Science*, 37(3), 359–373. <https://doi.org/10.1007/s00271-018-0610-z>
- BurchardLevine, V., Nieto, H., Riaño, D., Kustas, W. P., Migliavacca, M., ElMadany, T. S., Nelson, J. A., Andreu, A., Carrara, A., Beringer, J., Baldocchi, D., & Martín, M. P. (2021). A remote sensingbased threesource energy balance model to improve global estimations of evapotranspiration in semiarid treegrass ecosystems. *Global Change Biology*, 28(4), 1493–1515. <https://doi.org/10.1111/gcb.16002>
- Campbell, G. (1990). Derivation of an angle density function for canopies with ellipsoidal leaf angle distributions. *Agricultural and Forest Meteorology*, 49(3), 173–176. [https://doi.org/10.1016/0168-1923\(90\)90030-a](https://doi.org/10.1016/0168-1923(90)90030-a)
- Campbell, G. S., & Norman, J. M. (1998). The Light Environment of Plant Canopies. In *An Introduction to Environmental Biophysics* (pp. 247–278). Springer New York. https://doi.org/10.1007/978-1-4612-1626-1_15
- Féret, J.-B., Gitelson, A., Noble, S., & Jacquemoud, S. (2017). PROSPECT-D: Towards modeling leaf optical properties through a complete lifecycle. *Remote Sensing of Environment*, 193, 204–215. <https://doi.org/https://doi.org/10.1016/j.rse.2017.03.004>
- Guzinski, R., Nieto, H., Barrios, J. M., Ghariani, W., Gellens-Meulenberghs, F., De Pue, J., & Lacaze, R. (2025). Towards a global actual evapotranspiration product for the Copernicus Land Monitoring Service. <https://doi.org/10.5194/egusphere-2025-4342>
- Guzinski, R., Nieto, H., Sanchez, J. M., Lopez-Urrea, R., Boujnah, D. M., & Boulet, G. (2021). Utility of Copernicus-Based Inputs for Actual Evapotranspiration Modeling in Support of Sustainable Water Use in Agriculture. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 14, 11466–11484. <https://doi.org/10.1109/JSTARS.2021.3122573>
- Kustas, W. P., & Norman, J. M. (1999). Evaluation of soil and vegetation heat flux predictions using a simple two-source model with radiometric temperatures for partial canopy cover. *Agricultural and Forest Meteorology*, 94(1), 13–29. [https://doi.org/10.1016/s0168-1923\(99\)00005-2](https://doi.org/10.1016/s0168-1923(99)00005-2)
- Kustas, W. P., Anderson, M. C., Alfieri, J. G., Knipper, K., Torres-Rua, A., Parry, C. K., Nieto, H., Agam, N., White, W. A., Gao, F., McKee, L., Prueger, J. H., Hipps, L. E., Los, S., Alsina, M. M., Sanchez, L., Sams, B., Dokoozlian, N., McKee, M., ... Hain, C. (2018). The Grape Remote Sensing Atmospheric Profile and Evapotranspiration Experiment. *Bulletin of the American Meteorological Society*, 99(9), 1791–1812. <https://doi.org/10.1175/bams-d-16-0244.1>
- Luo, Y., El-Madany, T. S., Filippa, G., Ma, X., Ahrens, B., Carrara, A., Gonzalez-Cascon, R., Cremonese, E., Galvagno, M., Hammer, T. W., Pacheco-Labrador, J., Martín, M. P., Moreno, G., Perez-Priego, O., Reichstein, M., Richardson, A. D., Rörmann, C., & Migliavacca, M. (2018). Using Near-Infrared-Enabled Digital Repeat Photography to Track Structural and Physiological Phenology in Mediterranean Tree–Grass Ecosystems. *Remote Sensing*, 10(8), 1293. <https://doi.org/10.3390/rs10081293>
- Massman, W., Forthofer, J., & Finney, M. (2017). An improved canopy wind model for predicting wind adjustment factors and wildland fire behavior. *Canadian Journal of Forest Research*, 47(5), 594–603. <https://doi.org/10.1139/cjfr-2016-0354>
- Moore, C. E., Beringer, J., Evans, B., Hutley, L. B., McHugh, I., & Tapper, N. J. (2016). The contribution of trees and grasses to productivity of an Australian tropical savanna. *Biogeosciences*, 13(8), 2387–2403. <https://doi.org/10.5194/bg-13-2387-2016>
- Nieto, H., Kustas, W. P., Alfieri, J. G., Gao, F., Hipps, L. E., Los, S., Prueger, J. H., McKee, L. G., & Anderson, M. C. (2018). Impact of different within-canopy wind attenuation formulations on modelling sensible heat flux using TSEB. *Irrigation Science*, 37(3), 315–331. <https://doi.org/10.1007/s00271-018-0611-y>
- Norman, J., Kustas, W., & Humes, K. (1995). Source approach for estimating soil and vegetation energy fluxes in observations of directional radiometric surface temperature. *Agricultural and Forest Meteorology*, 77(3), 263–293. [https://doi.org/https://doi.org/10.1016/0168-1923\(95\)02265-Y](https://doi.org/https://doi.org/10.1016/0168-1923(95)02265-Y)
- Parry, C. K., Nieto, H., Guillevic, P., Agam, N., Kustas, W. P., Alfieri, J., McKee, L., & McElrone, A. J. (2019). An intercomparison of radiation partitioning models in vineyard canopies. *Irrigation Science*, 37(3), 239–252. <https://doi.org/10.1007/s00271-019-00621-x>
- Pastorello, G., Trotta, C., Canfora, E., Chu, H., Christianson, D., Cheah, Y.-W., Poindexter, C., Chen, J., Elbashandy, A., Humphrey, M., Isaac, P., Polidori, D., Reichstein, M., Ribeca, A., Ingen, C. van, Vuichard, N., Zhang, L., Amiro, B., Ammann, C., ... Papale, D. (2020). The FLUXNET2015 dataset and the ONEFlux processing pipeline for eddy covariance data. *Scientific Data*, 7(1). <https://doi.org/10.1038/s41597-020-0534-3>
- Potapov, P., Li, X., Hernandez-Serna, A., Tyukavina, A., Hansen, M. C., Kommareddy, A., Pickens, A., Turubanova, S., Tang, H., Silva, C. E., Armston, J., Dubayah, R., Blair, J. B., & Hofton, M. (2021). Mapping global forest canopy height through integration of GEDI and Landsat data. *Remote Sensing of Environment*, 253, 112165. <https://doi.org/10.1016/j.rse.2020.112165>
- PRIESTLEY, C. H. B., & TAYLOR, R. J. (1972). On the Assessment of Surface Heat Flux and Evaporation Using Large-Scale Parameters. *Monthly Weather Review*, 100(2), 81–92. [https://doi.org/10.1175/1520-0493\(1972\)100<0081:otaosh>2.3.co;2](https://doi.org/10.1175/1520-0493(1972)100<0081:otaosh>2.3.co;2)
- Principles of Environmental Physics*. (2013,). <https://doi.org/10.1016/c2010-0-66393-0>
- Raupach, M. R. (1994). Simplified expressions for vegetation roughness length and zero-plane displacement as functions of canopy height and area index. *Boundary-Layer Meteorology*, 71(1–2), 211–216. <https://doi.org/10.1007/bf00709229>
- Schaudt, K. (2000). An approach to deriving roughness length and zero-plane displacement height from satellite data, prototyped with BOREAS data. *Agricultural and Forest Meteorology*, 104(2), 143–155. [https://doi.org/10.1016/s0168-1923\(00\)00153-2](https://doi.org/10.1016/s0168-1923(00)00153-2)
- Shuttleworth, W. J., & Wallace, J. S. (1985). Evaporation from sparse crops-an energy combination theory. *Quarterly Journal of the Royal Meteorological Society*, 111(469), 839–855. <https://doi.org/https://doi.org/10.1002/qj.49711146910>
- Verhoef, W., Jia, L., Xiao, Q., & Su, Z. (2007). Unified Optical-Thermal Four-Stream Radiative Transfer Theory for Homogeneous Vegetation Canopies. *IEEE Transactions on Geoscience and Remote Sensing*, 45(6), 1808–1822. <https://doi.org/10.1109/TGRS.2007.895844>

Weiss, A., & Norman, J. (1985). Partitioning solar radiation into direct and diffuse, visible and near-infrared components. *Agricultural and Forest Meteorology*, 34(2–3), 205–213. [https://doi.org/10.1016/0168-1923\(85\)90020-6](https://doi.org/10.1016/0168-1923(85)90020-6)